



CONTEXT AWARE INTELLIGENT IT SUPPORT SYSTEMS:

A Multi-Agent Framework with Risk-Adaptive and Verifiable Remediation

Complete Draft Submission Version 01

BSc (Honours) in Software Engineering

Faculty of Computing

NSBM Green University

10 September 2026

Name : A.P.T.N. Pathirana

Index : 28647

CONTENTS

Chapter – 01 INTRODUCTION	4
1.1 Chapter Overview.....	4
1.2 Problem Background.....	4
1.3 Problem Statement.....	5
1.4 Research Question	5
1.5 Research Motivation.....	6
1.6 Research Aim.....	6
1.7 Research Objectives	6
1.8 Rich picture of the proposed solution.....	7
1.9 Resource Requirements	9
1.10 Project scope	10
1.11 Chapter Summary.....	11
Chapter – 02 LITERATURE REVIEW	11
2.1 Chapter overview	11
2.2 Conceptual Map of the literature.....	11
2.3 Domain Overview	12
2.4 Existing Systems / Frameworks / Designs	13
2.5 Technological Analysis	17
2.6 Reflection	22
Chapter – 03 METHODOLOGY	23
3.1 Research Paradigm.....	23
3.2 Research Approach.....	23
3.3 Research Strategy.....	24
3.4 Fact Collection Mechanisms	24
3.5 Research Methodology Execution Workflow	26
3.6 Project Management Methodology	30
3.7 Chapter Summary.....	31
Chapter – 04 SYSTEM REQUIREMENT SPECIFICATION	32
4.1 Chapter Overview	32
4.2 Stakeholder Analysis.....	32
4.3 Operationalization Process.....	32
4.4 System/Model Analysis.....	33

4.5 Proposed System Architecture	38
4.6 Functional and Non-functional Requirements	38
4.7 Chapter Summary.....	40
Chapter – 05 IMPLEMENTATION AND DESIGN	40
5.1 Chapter Overview	40
5.2 Framework Implementation Process.....	40
5.3 Core Algorithms and Logic.....	41
5.4 Framework Workflow and Component Integration	44
5.5 Technology Selection and Justification.....	44
5.6 Significant Implementation Attempts	45
5.7 Chapter Summary.....	46
Chapter – 06 TESTING AND EVALUATION	47
6.1 Chapter Overview	47
6.2 Test Plan and Test Cases	47
6.3 Testing and Evaluation Workflow	48
6.4 Review of Test Strategies Used.....	50
6.5 Chapter Summary.....	54
Chapter – 07 CONCLUSIONS AND RECOMMENDATIONS.....	54
7.1 Accomplishment of the Research Objectives.....	55
7.2 Problems Encountered	56
7.3 Self-Reflection	56
7.4 Business Insight of AutoOps AI.....	57
7.5 Real-World Application Possibilities	57
7.6 Future Recommendations.....	58
7.7 Final Summary	60
REFERENCES	60

TABLES

Table 1.1 The Five Specialized Agents and Their Roles.....	8
Table 2.1. Comparative assessment of existing systems and their suitability for AutoOps AI	17
Table 3.1. Summary of methodological choices and justification	24
Table 4.1. Stakeholder needs and system responsibilities.....	32
Table 5.1. Operationalisation of the five risk factors	42
Table 6.1. Principal functional and safety test cases	48
Table 7.1. Accomplishment of the research objectives	55

FIGURES

Figure 1.1. Proposed AutoOps AI risk-adaptive and verifiable remediation workflow	7
Figure 2.1. Conceptual map of the literature supporting AutoOps AI	12
Figure 3.1. AutoOps AI Design Science Research execution workflow.....	26
Figure 4.1. Main AutoOps AI use cases	34
Figure 5.1. AutoOps AI implementation pipeline	41
Figure 6.1. AutoOps AI testing and evaluation workflow.....	49

Chapter – 01 INTRODUCTION

1.1 Chapter Overview.

This chapter introduces AutoOps AI, a context-aware intelligent IT-support framework that combines Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), specialised AI agents and controlled remediation. The chapter first explains the practical difficulty of resolving common end-user IT problems when user descriptions are incomplete, organisational knowledge is distributed across documents, and automated actions can create additional risk. It then narrows this broad problem to the central research gap: the limited integration of evidence-grounded troubleshooting, contextual risk assessment, proportional human approval, pre-action and post-action verification, rollback, escalation and auditability within one end-user support framework.

The chapter states the research question, motivation, aim and objectives and then presents the proposed workflow, resource requirements and scope. Throughout this thesis, remediation means an action intended to correct a diagnosed problem. Verifiable remediation means that the system checks whether the required conditions exist before execution and whether the original problem has actually been resolved afterward. Therefore, the research is not concerned only with whether an LLM can recommend or execute a command. It examines whether an AI-supported remediation process can remain useful while its authority is adjusted according to risk and its result is supported by observable evidence.

1.2 Problem Background

Organisations depend on endpoint devices, networks, applications and cloud services for routine work. When an employee experiences a problem such as failed network access, a stopped service, an application error or insufficient temporary storage, the normal support process involves reporting the issue, waiting for a support officer, repeating diagnostic information and following several troubleshooting steps. A large proportion of this work is knowledge-intensive and repetitive. AutoTSG demonstrated that troubleshooting guides are widely used in large-scale incident operations, while also showing that they can be fragmented, incomplete and difficult to translate into executable workflows [1]. These limitations can delay resolution and increase dependence on the availability and experience of individual support personnel.

Recent LLM-based agents create an opportunity to improve this process. ReAct showed that language models can interleave reasoning with actions, rather than producing text alone [2]. Multi-agent frameworks such as AutoGen further demonstrate that specialised agents, tools and human participants can be arranged into collaborative workflows [3]. Research on LLM-based multi-agent systems identifies role specialisation and structured communication as useful for decomposing complex tasks, although coordination, reliability and evaluation remain open challenges [4]. In IT operations, RCAGENT illustrates how a tool-augmented agent can gather evidence and analyse operational problems, indicating that LLMs can support more active troubleshooting than a conventional question-answering chatbot [5].

AutoOps AI applies these capabilities to common end-user IT support. A conversation agent interprets the user's request and maintains context. Retrieval-Augmented Generation (RAG) retrieves relevant organisational procedures and previously approved knowledge so that the diagnosis and proposed solution are supported by evidence. The quality of this retrieval must be evaluated because an answer can appear plausible even when the retrieved context is irrelevant or used unfaithfully. RAGAS therefore evaluates dimensions such as context relevance, faithfulness and answer relevance [6]. Ticket, image-analysis and status-management functions can support the wider service workflow, while a dedicated action component can perform selected diagnostic or remediation operations.

However, connecting an LLM to tools changes the nature of the risk. An incorrect chat response may confuse a user, but an incorrect tool call can alter system state. Empirical work on LM agents has identified failures that can produce undesirable real-world outcomes even when the model can describe the relevant risk [7]. AgentDojo also demonstrates that tool-using agents can be influenced by untrusted content and prompt-injection attacks, creating a conflict between completing a requested task and maintaining security [8]. Consequently, evidence-grounded diagnosis alone is insufficient. The system must control what can be executed, assess the risk of the proposed effect, involve a human when necessary and verify the outcome after execution.

1.3 Problem Statement

1.3.1 General Problem

Conventional IT-support processes often rely on queues, manual diagnosis and repeated execution of standard troubleshooting instructions. This approach can be slow for users and inefficient for support teams, particularly when the relevant solution already exists in organisational documentation. LLMs, RAG and agent-based automation can reduce this burden, but an organisation cannot safely grant unrestricted system access to a probabilistic language model. Incorrect diagnosis, incomplete context, weak evidence, excessive permissions or a misunderstood command may cause an action to fail or create a new problem. The general problem is therefore a tension between efficiency and control: keeping every action manual limits the value of automation, while allowing every AI-proposed action to execute creates unacceptable operational and security risk.

1.3.2 Specific Problem

Existing studies contribute important elements but do not fully resolve this tension for common end-user IT support. IT troubleshooting research automates guides or gathers diagnostic evidence [1], [5] RAG supports knowledge grounding and measurable faithfulness [6] multi-agent frameworks distribute responsibilities [3], [4] and agent-safety studies detect risky behaviour or enforce runtime rules [7], [8], [9]. NIST's AI Risk Management Framework emphasises that risk should be governed, mapped, measured and managed throughout the AI lifecycle [10], while its Generative AI Profile extends these considerations to risks specific to generative systems [11]. Nevertheless, these elements still require integration into a domain-specific closed-loop remediation process.

The specific gap addressed by this research is the absence of an evaluated IT-support framework in which contextual diagnosis and organisational evidence are linked to a structured, action-level risk decision and verifiable execution. Before a proposed action is authorised, AutoOps AI must consider its possible impact, diagnostic confidence, evidence quality, reversibility and affected resources. The result must be translated into a low, medium or high risk class that determines whether the action may proceed, requires user or IT-expert approval, or must be blocked or escalated. Furthermore, execution success must be judged against expected postconditions rather than only a successful command return code. Runtime-enforcement research such as AgentSpec supports the value of explicit and interpretable execution constraints [9], while recent work on verifiably safe tool use argues for enforceable safety specifications over agent capabilities and tool sequences [12]. AutoOps AI adapts these principles to a practical and testable framework for end-user IT remediation.

1.4 Research Question

Primary Research Question:

How can a context-aware, multi-agent IT-support system combine RAG-grounded troubleshooting with risk-adaptive, human-gated and verifiable remediation to resolve common end-user IT problems safely and effectively?

The primary question is supported by the following sub-questions:

How can conversational context and organisational knowledge be used to produce evidence-grounded diagnoses and remediation proposals?

How can impact, diagnostic confidence, evidence quality, reversibility and affected-resource scope be combined to classify a proposed action as low, medium or high risk?

How can the risk class and organisational policy determine whether an action proceeds, requires user or IT-expert approval, or is blocked and escalated?

How can pre-action and post-action verification determine whether an authorised remediation is safe to start and successful after execution?

To what extent does the proposed framework improve safe resolution, traceability and efficiency compared with a non-risk-adaptive approach?

1.5 Research Motivation

The practical motivation for this research is the repeated nature of many support requests and the delay produced when every case must wait for manual intervention. A system that can understand a user's description, retrieve an approved procedure and perform a suitable low-risk action could reduce unnecessary effort and help users return to work sooner. However, speed is not a sufficient outcome if the same automation can make unauthorised or damaging changes.

The academic motivation is therefore to investigate controlled autonomy rather than maximum autonomy. Existing work confirms both the usefulness of agentic troubleshooting and the hazards of tool use [5], [7], [8]. Human-AI interaction guidelines also emphasise communicating system capabilities, supporting efficient correction and allowing users to control or recover from AI behaviour [13]. These findings motivate a framework that varies human involvement according to risk and maintains evidence, approval records, execution results and recovery information for accountability.

1.6 Research Aim

The aim of this research is to design, develop and evaluate AutoOps AI, a context-aware multi-agent IT-support framework that combines RAG-grounded troubleshooting with risk-adaptive, human-gated and verifiable remediation to improve the safety, reliability and effectiveness of resolving common end-user IT problems within a controlled environment.

1.7 Research Objectives

1.7.1 To Identify

To identify the operational limitations of conventional IT-support workflows and the safety, reliability and governance risks that arise when LLM-based agents are permitted to use diagnostic and remediation tools.

1.7.2 To Analyze

To analyse recent research on LLM agents, multi-agent collaboration, RAG, automated troubleshooting, AI risk management, human oversight, runtime policy enforcement and verifiable tool use, and to derive suitable requirements for controlled IT remediation.

1.7.3 To Design and Develop

To design and develop AutoOps AI with specialised agents for conversational issue understanding, ticket intelligence, evidence retrieval, image-assisted analysis, action control and ticket status management, together with a remediation safety layer that performs structured risk classification, approval routing, precondition validation, controlled execution, postcondition verification, audit logging, rollback and escalation.

1.7.4 To evaluate

To evaluate the framework in controlled end-user IT-support scenarios using measures such as risk-classification correctness, unsafe-action prevention, remediation success, verification correctness, approval and escalation appropriateness, RAG relevance, execution trace completeness, resolution time, usability and user trust.

1.8 Rich picture of the proposed solution

The proposed solution separates reasoning from authority. The conversational and diagnostic components may propose a remediation plan, but they cannot directly execute unrestricted commands. A dedicated safety layer evaluates the intended effect using contextual and organisational evidence. Low-risk actions may proceed under configured policy; medium-risk actions require explicit human approval; and high-risk actions require IT-expert approval or are blocked and escalated. Every permitted action must satisfy preconditions, run through an approved action catalogue and produce auditable evidence for post-action verification.

1. USER ISSUE: User describes the problem or provides an error image.
2. CONTEXT AND EVIDENCE: Specialised agents interpret the issue; RAG retrieves approved organisational knowledge.
3. DIAGNOSIS AND PLAN: The system creates an evidence-linked diagnosis and proposes an approved action type.
4. RISK-ASSESSMENT GATE: Assess impact, confidence, evidence quality, reversibility and affected resources.
5. POLICY DECISION: LOW: controlled execution MEDIUM: user approval HIGH: IT approval, block or escalation.
6. PRE-ACTION VERIFICATION: Validate permissions, system state, parameters, scope and rollback availability.
7. CONTROLLED EXECUTION: Run only the authorised action through the restricted Action Executor.
8. POST-ACTION VERIFICATION: Compare actual results with expected postconditions and confirm issue resolution.
9. CLOSE, ROLLBACK OR ESCALATE: Record the complete trace; close on success or recover/escalate on failure.

Figure 1.1. Proposed AutoOps AI risk-adaptive and verifiable remediation workflow

The workflow is closed-loop because the outcome of an action returns to the support process. A failed postcondition does not produce automatic ticket closure. Instead, the system records the evidence, attempts an approved rollback where available and escalates the unresolved case. This makes verification,

recovery and traceability part of the remediation design rather than optional activities performed after automation.

The Five Specialized Agents and Their Roles

Agent	Responsibility	Key Technologies
LLM Conversation Agent	Natural language understanding, multi-turn dialogue management, message classification (technical/general/greeting), contextual response generation, escalation detection	Google Gemini LLM, Prompt Engineering
Ticket Intelligence Agent	Intelligent ticket creation timing, priority analysis (urgency scoring 0–10), LLM-powered title/description generation, category classification, resolution detection	Google Gemini LLM, Rule-based + LLM hybrid
Action Executor Agent	Whitelisted system action management (50+ safe operations), parameter validation, user permission workflow, PowerShell command execution, follow-up action suggestions	PowerShell, NLP Pattern Matching, Risk Assessment
Image Analysis Agent	Visual analysis of error screenshots and device photos, OCR text extraction, issue categorization from images, keyword extraction for RAG search	Google Gemini Vision API
Ticket Status Agent	Ticket lifecycle management (OPEN → IN PROGRESS → RESOLVED → CLOSED), resolution detection, re-open detection, SLA tracking, auto closure, abandonment detection	Rule based state machine, SLA algorithms

Table 1.1 The Five Specialized Agents and Their Roles

1.9 Resource Requirements

1.9.1 Hardware Requirements

Resource	Minimum requirement	Purpose
Processor	Modern 64-bit multi-core processor	Runs the web application, local services and controlled experiments
Memory	8 GB RAM; 16 GB recommended	Supports the backend, frontend, vector store and concurrent agents
Storage	At least 20 GB available SSD space	Stores project files, knowledge documents, test data and logs
Network	Stable internet connection	Provides access to the selected hosted LLM API and research resources
Test environment	Isolated Windows virtual machine or test endpoint	Allows safe execution and verification of approved remediation actions

Table 1.2 Hardware requirements

1.9.2 Software Requirements.

Category	Selected technology	Purpose
Backend	Python and FastAPI	Agent orchestration, APIs, risk controls and experiment logging
Frontend	React with Vite	User conversation, approvals, tickets and administrative views
Knowledge retrieval	ChromaDB and embedding model	Stores and retrieves organisational knowledge for RAG
Operational data	SQLite / relational database	Stores users, tickets, approvals, actions and audit records
AI service	Selected LLM API	Natural-language interpretation, grounded response synthesis and planning
Action environment	Controlled Windows commands and scripts	Executes allow-listed diagnostic and remediation actions
Development	Git, GitHub, Docker and API-testing tools	Version control, reproducibility, deployment and verification

Table 1.3. Software requirements

1.10 Project scope

The study focuses on common end-user IT-support scenarios executed in a controlled test environment. AutoOps AI supports text-based issue reporting, context-aware conversation, ticket classification, retrieval from curated organisational knowledge, evidence-grounded diagnosis, low/medium/high risk classification, policy-based human approval, allow-listed remediation, pre-action and post-action verification, audit logging, rollback where technically feasible, and escalation to a human specialist. Evaluation covers both individual components and the complete workflow.

In scope	Out of scope
Common end-user device, application, storage, service and network-support scenarios	Enterprise-wide infrastructure orchestration and large-scale production deployment
LLM-assisted conversation and specialised multi-agent or modular coordination	Training a new foundation language model
RAG using approved organisational and technical documents	Unrestricted web search as an authority for automated actions
Low, medium and high risk classification using contextual factors	Autonomous execution of destructive, illegal or unapproved commands
Risk-based human approval, blocking and expert escalation	Removing organisational administrators or human support personnel
Allow-listed actions in an isolated Windows test environment	Uncontrolled remote access to real organisational endpoints
Pre-action checks, post-action outcome verification, audit and recovery	Predictive issue detection, predictive maintenance, SLA-breach prediction and 24/48/72-hour forecasting
Controlled technical experiments and a supporting user survey	Long-term production trials across multiple organisations

Table 1.4. Project scope

The prototype focuses on common end-user Windows problems that can be represented with measurable preconditions and postconditions. High-impact, destructive or insufficiently understood operations remain prohibited. Predictive issue detection is excluded so that development and evaluation remain centred on the research problem of safely controlling and verifying remediation after a user reports an issue. Prediction may be investigated in future work only after the present control framework has been evaluated.

1.11 Chapter Summary

This chapter established that AI-assisted troubleshooting can improve access to organisational knowledge and reduce repetitive support effort, but tool enabled remediation introduces risks that cannot be addressed by conversational quality alone. AutoOps AI is therefore designed around controlled autonomy. It uses specialised agents and RAG to understand the problem and create an evidence-grounded plan, while a separate remediation safety layer assesses impact, confidence, evidence quality, reversibility and affected resources. The resulting low, medium or high risk classification determines whether the action proceeds, requires human approval, or is blocked and escalated. Pre-action validation, post-action verification, audit logging and rollback complete the closed loop. The chapter defined one primary research question, five supporting questions, four research objectives and a controlled scope that excludes predictive analytics. Chapter 2 critically examines the literature that supports these design choices and identifies the detailed research gap addressed by the proposed framework.

Chapter – 02 LITERATURE REVIEW

2.1 Chapter overview

This chapter critically reviews research that supports the design of AutoOps AI, a context-aware multi-agent IT support framework with risk adaptive and verifiable remediation. The review is organised around two connected questions. The first is whether LLM agents, retrieval-augmented generation (RAG), and multi-agent coordination can improve the understanding and resolution of IT-support incidents. The second, and more important question for this research, is how an AI-generated remediation proposal can be governed before, during, and after execution. The chapter therefore examines both capability-oriented work, such as automated troubleshooting and agentic root-cause analysis, and safety-oriented work, such as risk assessment, runtime enforcement, human approval, precondition checking, postcondition verification, rollback, and auditability.

The reviewed studies were published between 2019 and 2026 and include peer-reviewed conference papers, established risk-management guidance, and recent work on safe tool-using agents. The analysis does not treat a high task-completion score as sufficient evidence of operational safety. Instead, each approach is evaluated according to its relevance to end-user IT support, the evidence it uses, the authority granted to the agent, the controls applied to proposed actions, and the method used to establish that an executed fix achieved the intended outcome. Predictive issue detection and predictive analytics are outside the present research scope; they are not analysed as part of the proposed solution and may only be considered in future work.

2.2 Conceptual Map of the literature

The literature can be understood as five intersecting areas. IT-support and AIOps research explains the operational setting and the need to reduce repetitive diagnosis and remediation work. LLM agent research explains how a model can reason, plan, call tools, and collaborate with specialised agents. RAG research explains how organisational documents can ground a response and provide traceable evidence. AI-risk and human-interaction research explains why uncertain automated decisions require governance, communication, and appropriate human control. Finally, agent safety and software-verification research

explains how tool calls can be intercepted, constrained, checked, logged, and reversed. Figure 2.1 shows these relationships and separates literature that mainly improves capability from literature that mainly controls execution.

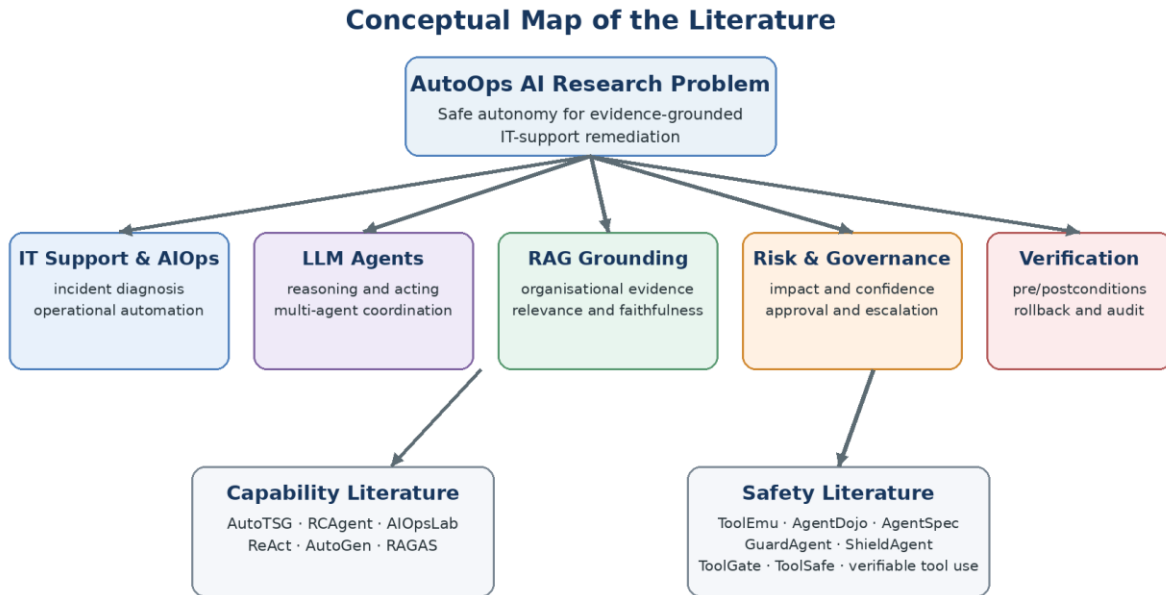


Figure 2.1. Conceptual map of the literature supporting AutoOps AI

No single area is sufficient by itself. A capable diagnostic agent without organisational grounding may produce unsupported advice. A grounded answer without action control may still execute an inappropriate command. A fixed rule engine may block known violations but cannot determine whether the diagnosis is well supported. Conversely, asking a human to approve every action removes much of the practical benefit of automation. AutoOps AI therefore adopts an integrated view: evidence informs confidence, context informs impact and scope, risk determines the required authority, and explicit checks determine whether execution may begin and whether it succeeded.

2.3 Domain Overview

IT support covers the reporting, diagnosis, resolution, documentation, and closure of technical problems affecting users and organisational services. Traditional support is commonly ticket based, a user describes a problem, the request is categorised and prioritised, a technician gathers additional information, and a solution is selected from experience, documentation, or escalation paths. This workflow provides accountability but can be slow when queues are long or when information must be collected repeatedly. It also depends heavily on the quality and accessibility of troubleshooting knowledge.

AutoTSG provides strong evidence that troubleshooting guides are valuable in real incident operations. Its study of more than 4,000 guides linked to thousands of incidents found that such guides can reduce mitigation effort, while feedback from more than 400 engineers revealed problems including fragmentation, incomplete steps, validation difficulties, and maintenance cost [1]. The proposed framework converted natural-language guides into executable workflows and achieved 0.89 accuracy for identifying guide statements, with 0.94 precision and 0.91 recall for parsing executable elements [1]. This

demonstrates that operational knowledge can be transformed into structured actions. However, guide automation assumes that the underlying guide and extracted command are suitable for the current state. It does not provide the complete action level risk classification, human-gating, and post-remediation verification required by AutoOps AI.

AIOps extends IT operations by applying data analysis and automation to incident detection, localisation, diagnosis, and mitigation. Recent LLM agents broaden this concept because they can interpret unstructured reports and interact with tools. AIOpsLab presents a realistic environment for evaluating such agents across the incident lifecycle, including fault injection, workload generation, telemetry collection, and agent-cloud interaction [14]. Its contribution is especially relevant to evaluation: an operational agent should be tested in a dynamic environment where actions change system state, rather than only on a static question-answer dataset. Nevertheless, AIOpsLab focuses on cloud operations and benchmarking. AutoOps AI targets common end-user Windows support and adds an explicit safety decision before remediation.

The domain therefore contains a practical shift from advisory automation to operational autonomy. An advisory chatbot suggests steps that a user or technician performs. An agentic support system may inspect the machine and invoke a tool itself. This shift creates additional risks involving privilege, unintended side effects, inaccurate parameters, untrusted retrieved content, and incorrect assumptions about current state. The literature consequently supports a controlled-autonomy model in which the AI may analyse broadly but may act only through narrow, policy-governed capabilities.

2.4 Existing Systems / Frameworks / Designs

2.4.1 AutoTSG and Executable Troubleshooting Knowledge

AutoTSG combines machine learning and program synthesis to transform troubleshooting guides into executable workflows [1]. The work is suitable for the knowledge to action part of AutoOps AI because it shows how unstructured operational instructions can be identified, parsed, validated, and presented in a repeatable form. It also demonstrates the importance of preserving human-understandable workflow structure rather than producing an unexplained command. However, AutoTSG is centred on guide conversion for cloud incident response. Its evaluation does not establish whether an action should receive different approval requirements according to impact, evidence quality, reversibility, and affected-resource scope. AutoOps AI therefore adopts the idea of an approved action catalogue but places a separate risk and verification layer between a retrieved procedure and its execution.

2.4.2 ReAct, AutoGen and Multi-Agent Coordination

ReAct introduced a prompting approach in which an LLM interleaves reasoning traces with actions and observations [2]. This pattern is relevant because IT troubleshooting is iterative: an initial hypothesis should lead to a diagnostic check, and the returned observation should update the next decision. Its weakness for remediation is that reasoning remains probabilistic and the same agent may both decide and act. A plausible reasoning trace does not guarantee that the selected action is authorised or safe.

AutoGen provides a framework for applications built from conversations among multiple agents, tools, and human participants [3]. The wider survey by Guo et al. identifies role specialisation, communication, planning, and coordination as central elements of LLM-based multi-agent systems, but also highlights unresolved reliability and evaluation challenges [4]. These findings support AutoOps AI's use of specialised agents for conversation, retrieval, diagnosis, action control, and ticket management.

Specialisation can reduce prompt complexity and make responsibilities visible. However, adding agents does not automatically improve correctness; agents can repeat or amplify the same unsupported assumption. For this reason, AutoOps AI treats multi-agent agreement as supporting information rather than as independent proof. Execution authority remains with the safety controller and policy layer.

2.4.3 RCAgent and Agentic Root-Cause Analysis

RCAgent uses a tool-augmented autonomous agent to collect operational evidence and perform root-cause analysis in cloud systems [5]. It incorporates context-management and self-consistency mechanisms and was evaluated on industrial cloud scenarios, where it outperformed a ReAct baseline across root-cause, solution, evidence, and responsibility-identification tasks [5]. This work is highly relevant to evidence-gathering and diagnostic planning in AutoOps AI. It demonstrates that an agent can actively inspect an environment instead of relying only on a user's initial description.

However, RCAgent's principal objective is diagnostic effectiveness in a cloud setting, not risk adaptive end-user remediation. A correct root cause hypothesis does not by itself answer whether the proposed fix is reversible, whether it affects one user or a shared service, or who must approve it. AutoOps AI extends the workflow from diagnosis to governed execution by making the proposed operational effect, rather than the natural-language response, the object of risk assessment.

2.4.4 AIOpsLab and Evaluation of Autonomous Operations

AIOpsLab addresses the lack of realistic evaluation environments for operational agents by integrating deployable cloud applications, fault injection, workloads, telemetry, and an agent cloud interface [14]. It created 100 operational problems and evaluated multiple agents across tasks in the incident lifecycle [14]. This is suitable as a methodological reference because it shows that an agent should be evaluated through interaction and observable outcomes. It also supports the use of fault scenarios and repeatable initial states.

Its direct suitability is limited by scale and domain. Cloud benchmarks use microservices, Kubernetes, and production style telemetry, whereas the AutoOps AI prototype focuses on controlled end-user Windows issues. More importantly, AIOpsLab evaluates whether agents can complete operational tasks but does not itself define the five-factor risk decision, approval policy, rollback decision, or action specific postconditions proposed in this research. AutoOps AI therefore uses the benchmark principle while designing smaller, measurable scenarios appropriate to an undergraduate prototype.

2.4.5 RAG and RAGAS

Lewis et al. introduced RAG as a method that combines a generative model with retrieved non-parametric knowledge, improving factuality and enabling access to updateable external information [15]. In organisational IT support, this is valuable because approved runbooks, policies, and device procedures may change more frequently than an LLM is retrained. RAG also makes it possible to present the document passages that support a proposed diagnosis.

Retrieval does not guarantee grounding. RAGAS evaluates a RAG pipeline through dimensions including context relevance, answer relevance, and faithfulness [6]. These measures are appropriate for evaluating whether AutoOps AI retrieved useful evidence and whether the generated explanation is supported by that evidence. Nevertheless, RAGAS evaluates answer quality rather than the operational safety of a system changing action. A highly faithful answer may still recommend a high-impact or irreversible action. AutoOps AI therefore uses evidence quality as one risk factor instead of treating it as permission to execute.

2.4.6 ToolEmu and AgentDojo

Ruan et al. developed ToolEmu, an LM-emulated sandbox for identifying risks before deploying tool-using agents [7]. The benchmark covers 36 toolkits and 144 test cases and shows that agents may produce unsafe outcomes even when they can verbally recognise risk [7]. ToolEmu is valuable for scenario generation and pre-deployment testing. Its limitations are also important: the emulator and evaluator are themselves model-based, test-case creation still involves human effort, and the studied threat model is not exhaustive [7]. It therefore supports risk discovery but cannot serve as the only runtime control for AutoOps AI.

AgentDojo provides 97 realistic tasks and 629 security test cases for evaluating prompt-injection attacks and defences in tool-using agents [8]. Its tool-filtering defence substantially reduced attack success when the benign task required fewer capabilities than the attacker, but the defence was less effective when legitimate and malicious goals required the same tools or when tool requirements could not be planned in advance [8]. The implication for AutoOps AI is that tool isolation and least privilege are necessary but not sufficient. The system must additionally validate action parameters, source trust, affected scope, and expected state changes at each execution decision.

2.4.7 GuardAgent, ShieldAgent, AgentSpec, ToolGate and ToolSafe

GuardAgent separates the protected agent from a guard agent that translates user-defined safety requests into executable checking code [16]. It achieved strong guardrail accuracy across healthcare access-control and web-agent safety benchmarks [16]. ShieldAgent similarly protects another agent, but constructs verifiable policy structures and reasons over complete action trajectories; its evaluation reported an average improvement of 11.3% over previous methods and 90.1% recall [17]. Both studies support the architectural decision to place an independent safety component between diagnosis and action. Their evaluations, however, concern broad agent environments rather than IT remediation, and their binary safe/unsafe decisions do not directly provide the low-, medium-, and high-risk approval levels required in this research.

AgentSpec defines a lightweight domain-specific language for triggers, predicates, and enforcement actions at runtime [9]. It prevented unsafe execution in more than 90% of evaluated code-agent cases and introduced only millisecond-level overhead [9]. This provides strong support for explicit, interpretable policy enforcement. Its authors note that deterministic checkpoints do not reason about long-term consequences across a trajectory [9]. AutoOps AI uses checkpoint enforcement for a bounded prototype while also evaluating affected scope and routing uncertain or high-risk actions to a human.

ToolGate formalises each tool using a precondition and postcondition contract and maintains an explicit trusted symbolic state [18]. This closely matches AutoOps AI's verification requirement: a tool should run only if its precondition is satisfied, and its result should update system state only when the postcondition is verified. ToolGate also reports limitations involving static evaluation environments, incomplete contracts, and limited treatment of multimodal or long collaborative tasks [18]. AutoOps AI applies the same principle in a narrower domain where preconditions and postconditions can be manually specified for selected Windows actions.

ToolSafe monitors each proposed tool invocation before execution and provides safety feedback to a ReAct-style agent [19]. Its TS-Flow design reduced harmful tool calls by approximately 65% while improving benign task completion under prompt-injection tests [19]. This supports step-level interception rather than checking only the final plan. However, the paper notes that an agent may ignore feedback appended to its context and that independently trained agent and guard models may be misaligned [19].

AutoOps AI therefore does not rely on natural-language feedback alone: a denied action is technically prevented by the action executor.

Finally, Doshi et al. argue that agent safety should move from model-based assurances to enforceable specifications derived from system-level hazard analysis [12]. Their four-tier structure of blocklists, mustlists, allowlists, and confirmation is particularly relevant to risk-adaptive autonomy. The work is an emerging design proposal rather than a domain-complete evaluated IT-support system. AutoOps AI operationalises the core idea through low, medium, and high risk decisions, explicit approval routes, pre/post conditions, and recovery behaviour.

System	Primary focus	Useful contribution	Limitation for this research	Adopted implication
AutoTSG [1]	Guide-to-workflow automation	Structured, reusable troubleshooting actions	No risk-adaptive approval or full outcome verification	Approved action catalogue
RCAgent [5]	Autonomous cloud RCA	Tool-based evidence gathering and diagnosis	Cloud diagnosis focus; no action-risk routing	Diagnostic and evidence agents
AIOpsLab [14]	Interactive AIOps benchmark	Repeatable faults, telemetry, measurable outcomes	Cloud scale; not a remediation safety controller	Scenario-based evaluation
RAGAS [6]	RAG evaluation	Relevance and faithfulness measures	Answer quality is not action safety	Evidence-quality signal
ToolEmu [7]	Risk discovery in emulated tools	Scalable pre-deployment safety testing	Model-based emulator/evaluator; limited threat scope	Adversarial safety scenarios
AgentDojo [8]	Prompt-injection benchmark	Security tasks and tool-isolation evidence	Isolation fails when legitimate and unsafe tools overlap	Least privilege plus source checks
GuardAgent [16]	Guardrail agent	Independent policy checking	Binary/domain-generic decisions	Separate safety controller
ShieldAgent [17]	Verifiable policy reasoning	Action-trajectory policy compliance	Not tailored to IT remediation risk levels	Policy-grounded explanations
AgentSpec [9]	Runtime policy DSL	Interpretable enforcement checkpoints at	Limited long-horizon consequence reasoning	Technical enforcement rules

ToolGate[18]	Tool contracts	Preconditions, postconditions, trusted state	Contract quality and dynamic environments remain challenges	Verifiable execution
ToolSafe [19]	Step-level guardrail	Pre-execution interception and feedback	Agent may ignore feedback	Hard interception before tool use

Table 2.1. Comparative assessment of existing systems and their suitability for AutoOps AI

2.5 Technological Analysis

To overcome the limitations of existing frameworks, recent literature has explored advanced AI paradigms. This section critically analyzes these technologies across algorithms, design, and workflow.

2.5.1 Algorithmic analysis

Context-Aware Issue Understanding

A context-aware support system must maintain information across conversational turns rather than treating every message independently. Relevant context includes the user's stated symptoms, device and operating-system information, previous diagnostic observations, ticket history, user role, current privileges, and the organisational environment. In a ReAct-style process, each observation may confirm or reject a hypothesis and determine the next safe diagnostic question [2]. Multi-agent role separation can support this process by assigning conversation management, retrieval, diagnosis, and policy checking to different components [3], [4].

The main algorithmic risk is context contamination. Incorrect early information can persist and affect later decisions, while untrusted retrieved text may attempt to redirect a tool-using agent. AutoOps AI should therefore distinguish user claims, observed system facts, retrieved organisational evidence, and inferred hypotheses. Only observed or policy-approved facts should populate the trusted execution state. Diagnostic confidence should increase when independent evidence supports the same explanation and decrease when observations conflict or required information is missing.

Retrieval and Evidence-Quality Assessment

The RAG pipeline contains document ingestion, chunking, embedding, indexing, query construction, retrieval, optional reranking, and grounded response generation. For IT support, document metadata should include source, version, applicable operating system, device group, privilege requirement, and approval status. This metadata is operationally important because a technically similar procedure may be unsuitable for the user's device or organisational policy. The foundational RAG work shows the value of combining generation with external knowledge [15], while RAGAS provides practical quality dimensions for evaluating the retrieved context and generated response [6].

AutoOps AI should not convert a RAG score directly into permission. Evidence quality is better treated as a composite signal. High-quality evidence is relevant to the reported symptom, originates from an approved and current source, supports the diagnosis, and contains a procedure applicable to the observed environment. If one of these conditions is weak, the action's risk should increase or execution should be

withheld. This design addresses an important difference between answering and acting: weak retrieval can be corrected conversationally, whereas a weakly supported remediation can change the system.

Diagnosis Confidence

Confidence should represent the strength of the diagnosis, not the fluency of the generated explanation. A practical prototype can calculate confidence from measurable indicators such as symptom match, number and consistency of diagnostic observations, agreement with retrieved procedures, absence of competing explanations, and completeness of required inputs. RCAgent’s use of self-consistency suggests that repeated or alternative reasoning paths can improve stability [5], but model agreement alone may reproduce shared bias. The final confidence signal should therefore combine model output with external evidence and direct system observations.

A threshold-based confidence policy is suitable for the proposed undergraduate implementation because it is interpretable and testable. For example, a diagnosis supported by an approved runbook and confirmed by a direct status check may reach the high-confidence band. A diagnosis based only on a vague user description remains low confidence. Low confidence should raise the risk level even when the proposed command is normally low impact, because executing the wrong low-impact action can still waste time, hide the real fault, or create misleading evidence.

Five-Factor Action-Risk Classification

The central algorithmic contribution of AutoOps AI is a structured action-level risk assessment. NIST AI RMF organises AI-risk work around governance, mapping, measurement, and management [10], while the Generative AI Profile emphasises risks such as confabulation, information security, human-AI configuration, and the need to evaluate controls in context [11]. These principles support a multi-factor decision rather than a single confidence threshold.

The proposed classifier evaluates five factors. Impact estimates the severity of an incorrect or failed change. Diagnostic confidence estimates whether the proposed fix addresses the actual cause. Evidence quality measures the relevance, authority, currency, and faithfulness of the supporting sources. Reversibility considers whether the previous state can be restored and whether a tested rollback procedure exists. Affected-resource scope considers both the number and criticality of users, devices, services, accounts, or data objects that could be changed. The factors are related but not interchangeable. A highly reversible action can still be inappropriate if evidence is weak, while a well-supported action can still require expert approval if it affects a shared critical resource.

Risk factor	Operational interpretation	Literature basis
Potential impact	Expected severity if the action is wrong or fails	NIST risk mapping and management [10], [11]
Diagnosis confidence	Strength and consistency of observed support for the root cause	Iterative reasoning and evidence gathering [2], [5]
Evidence quality	Relevance, authority, currency, and faithful use of retrieved sources	RAG and RAGAS [6], [15]
Reversibility	Ability and readiness to restore the prior	Human correction/recovery [13];

	state	verifiable tool use [12]
Affected resources	Number, privilege level, and criticality of resources in scope	Tool risk and capability isolation [7], [8]

Table 2.2. Literature-derived interpretation of the AutoOps AI risk factors

A deterministic rule-based classifier is preferable to allowing the LLM to select the final risk class without constraint. The LLM may extract candidate factor values and provide an explanation, but policy rules should calculate or validate the class. A weighted score can support consistent comparisons, while override rules handle unacceptable conditions. For example, an irreversible change to a critical shared resource should remain high risk even if other scores are favourable. Missing required evidence should prevent automatic execution. This hybrid approach preserves flexibility in language understanding while keeping authority in transparent software logic.

The resulting class must be tied to an action. Low risk represents bounded, reversible, well-supported operations with limited scope. Medium risk represents actions requiring informed user approval because they alter user-level configuration or have moderate uncertainty. High risk represents privileged, broad, difficult-to-reverse, security-sensitive, or insufficiently supported actions. High-risk actions require IT-expert approval or are blocked. This classification is intentionally conservative because false classification of a dangerous action as low risk has a greater operational cost than unnecessarily escalating a safe action.

Precondition and Postcondition Verification

Pre-action verification tests whether the conditions assumed by the plan are true immediately before execution. Typical checks include the operating system and tool version, current service or configuration state, available permissions, action parameters, affected-resource identity, dependency status, and rollback availability. ToolGate provides a clear theoretical basis by representing tools with precondition and postcondition contracts and allowing trusted state to evolve only through verified results [18]. AgentSpec further supports intercepting actions at runtime and enforcing explicit predicates [9].

Post-action verification determines whether the intended change actually occurred and whether the reported problem is resolved. A command returning exit code zero is not sufficient. For example, a command may complete while the target service remains unavailable or while the user's connectivity problem persists. A suitable postcondition therefore combines technical state checks with task-level confirmation. If postconditions fail, the system must not mark the ticket as resolved. It should record the result, invoke a pre-approved rollback when available, or escalate the case. This creates a closed loop rather than a one-way command pipeline.

2.5.2 Design Analysis

Separation of Reasoning and Authority

The most important architectural decision is that diagnostic agents can propose actions but cannot directly perform unrestricted changes. The safety controller receives a structured proposal containing the intended action, target, parameters, evidence, confidence, expected effect, preconditions, postconditions, and rollback method. It then applies risk and policy rules before passing an approved request to a restricted executor. This separation follows the independent-guard pattern demonstrated by GuardAgent and

ShieldAgent [16], [17] and reduces the chance that the same probabilistic model both recommends and authorises its own action.

The executor should expose only a small catalogue of typed actions rather than a general shell. Parameters should be validated against schemas and allowlists. The execution identity should have least privilege and should receive only the capability required for the approved action. AgentDojo’s findings show why this restriction is valuable, while also showing that tool filtering alone cannot solve every security problem [8]. Therefore, capability restriction is combined with context, risk, and contract checks.

Human-Gated Control and Explanation

Human involvement should be proportional to risk. Requiring approval for every diagnostic query would create delay and approval fatigue, while allowing all actions would remove meaningful oversight. Low-risk actions may proceed automatically under policy; medium-risk actions require the user to review the intended change and expected effect; high-risk actions require an IT expert or are blocked. This approach is consistent with the confirmation tier proposed for verifiable agent tool use [12].

The approval interface must communicate what will change, why the system recommends it, which evidence supports the diagnosis, what resources are affected, and whether rollback is available. Human-AI interaction guidelines emphasise setting expectations, supporting correction, showing contextually relevant information, and enabling users to recover from AI behaviour [13]. Approval should therefore be specific to the action and parameters, time-limited, and invalidated if the plan changes. A generic “continue” button without these details would provide consent without informed control.

Auditability, Rollback and Escalation

Audit logging is a cross-cutting design requirement. A complete trace should record the user request, contextual facts, retrieved document identifiers, diagnosis, confidence, risk-factor values, policy decision, approval identity and time, precondition results, exact action and parameters, executor output, postcondition results, rollback activity, and final ticket state. This supports debugging, evaluation, accountability, and later policy improvement. Logs should avoid unnecessarily storing secrets or sensitive data and should be protected from unauthorised modification.

Rollback must be designed with the action rather than improvised after failure. Each executable action should define whether it is reversible, what state must be captured beforehand, how restoration is performed, and how restoration is verified. If safe rollback is unavailable, the risk score should increase and automation may be prohibited. Escalation is the final safety mechanism for low confidence, failed verification, policy conflict, or unexpected system state. The IT expert should receive the accumulated evidence and execution trace so that automation reduces, rather than repeats, the diagnostic workload.

2.5.3 Workflow Analysis

The literature supports a closed-loop workflow containing understanding, grounding, risk assessment, authorisation, execution, verification, and recovery. ReAct and RCagent support iterative observation and diagnosis [2], [5]. RAG and RAGAS support evidence retrieval and evaluation [6], [15]. NIST guidance supports contextual risk mapping and control [10], [11]. AgentSpec, ToolGate, ToolSafe, GuardAgent, and ShieldAgent support independent interception, policy checking, and verifiable execution [9], [16]-[19]. Human-AI guidance supports understandable approval and recovery [13]. Figure 2.2 combines these findings into the safety-centred workflow adopted by AutoOps AI.

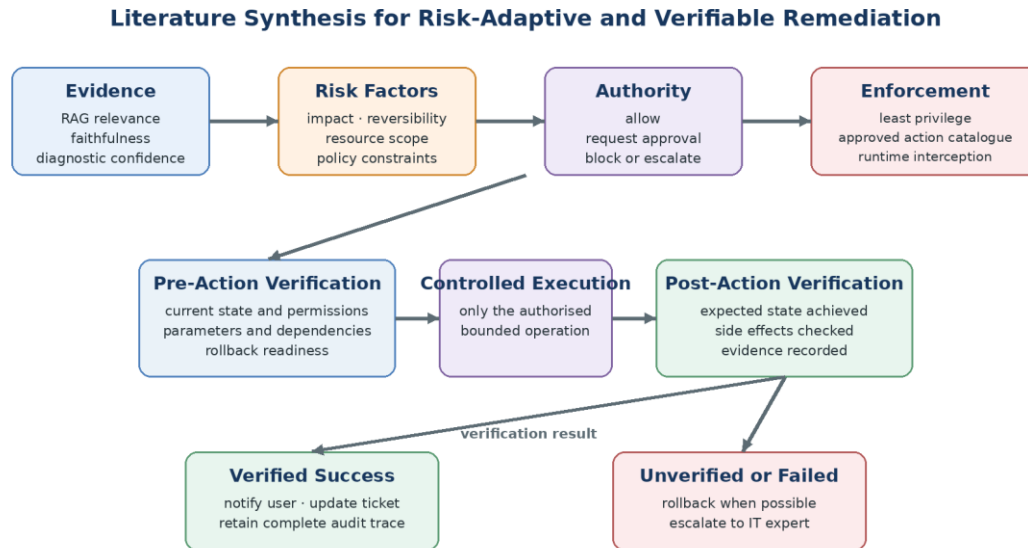


Figure 2.2. Literature synthesis for the AutoOps AI remediation safety workflow

The workflow begins with broad analytical freedom but progressively narrows authority. Agents may explore explanations and request non-destructive diagnostic observations. Once a system-changing action is proposed, the request becomes a structured transaction subject to risk rules. Approval does not bypass verification; it permits the executor to attempt one specific operation if its preconditions remain true. After execution, postconditions decide whether the transaction is accepted. This design prevents stale approval, unauthorised parameter changes, and false ticket closure.

The workflow also distinguishes three failure types. A planning failure occurs when diagnosis or evidence is insufficient; the system should ask for information or escalate without acting. An authorisation failure occurs when risk or policy does not permit the action; the system should block and explain the reason. An execution or verification failure occurs after an approved attempt; the system should preserve evidence, roll back when possible, and escalate. Treating these failures separately improves evaluation because a safe refusal should not be counted as a task failure in the same way as an unsafe execution.

Workflow stage	Supporting literature	AutoOps AI design implication
Understand issue and collect observations	ReAct [2], RCAgent [5]	Context record and diagnostic agent
Retrieve organisational evidence	RAG [15], RAGAS [6]	Source-aware retrieval and evidence scores
Assess action risk	NIST AI RMF [10], [11]; ToolEmu [7]	Five-factor risk assessment with overrides
Route authority	Human-AI guidelines [13];	Low/medium/high decision and approval

	verifiable tool use [12]	
Enforce permitted behaviour	AgentSpec [9], GuardAgent [16], ShieldAgent [17]	Independent safety controller and policy rules
Verify execution	ToolGate [18], ToolSafe [19]	Preconditions, restricted executor, postconditions
Recover and account	Human-AI guidelines [13]; NIST [10]	Rollback, escalation, ticket update, audit log

Table 2.3. Literature-to-design traceability for the proposed workflow

2.6 Reflection

The literature demonstrates meaningful progress, but it remains fragmented when considered from the perspective of common end-user IT remediation. Capability-oriented systems show that troubleshooting guides can be automated [1], agents can reason and act [2], multiple agents can collaborate [3], [4], and operational evidence can support root-cause analysis [5]. RAG provides external grounding [15], and RAGAS provides useful quality measures [6]. These contributions are necessary for AutoOps AI, but none establishes that a retrieved and plausible fix is safe to execute in the current organisational context. Safety-oriented studies address complementary parts of the problem. ToolEmu and AgentDojo reveal unsafe behaviour and prompt-injection risk [7], [8]. GuardAgent and ShieldAgent separate guardrail reasoning from the protected agent [16], [17]. AgentSpec enforces custom runtime constraints [9]. ToolGate formalises preconditions and postconditions [18], while ToolSafe checks individual tool invocations before execution [19]. Doshi et al. connect hazard analysis, formal specifications, capability control, and confirmation [12]. However, these works mainly evaluate general tool use, web agents, code agents, or cloud operations. They do not provide an evaluated end-user IT-support workflow that connects diagnosis evidence to a transparent risk class, approval route, verified remediation result, and recovery outcome.

Five specific gaps follow from this comparison. First, there is an integration gap: diagnosis, RAG, risk assessment, approval, execution, verification, and ticket handling are commonly studied as separate components. Second, there is a decision gap: existing guardrails often produce binary allow/deny results, whereas practical IT support needs graded autonomy that distinguishes low-, medium-, and high-risk actions. Third, there is an evidence-to-authority gap: RAG quality is normally evaluated as answer quality, not as one input to execution authority. Fourth, there is an outcome gap: successful tool invocation is often measured without requiring proof that the user's problem was resolved. Fifth, there is a domain-evaluation gap: current benchmarks do not directly measure risk-classification correctness, approval appropriateness, rollback behaviour, trace completeness, and safe-resolution performance for controlled end-user Windows scenarios.

AutoOps AI addresses these gaps through an integrated framework rather than by claiming that each individual mechanism is new. Its research contribution is the design and evaluation of a domain-specific control loop in which the proposed action is assessed using potential impact, diagnostic confidence, evidence quality, reversibility, and affected resources. The resulting risk class selects the permitted authority: controlled automatic continuation, user approval, IT-expert approval, or blocking and escalation. Execution is then conditioned on explicit preconditions and accepted only when

postconditions confirm the intended result. Rollback and escalation are part of the same workflow, and the complete process is auditable.

This position also defines the limits of the research. The prototype does not attempt unrestricted autonomous administration, universal formal verification, or guaranteed safety for every operating-system command. It concentrates on a curated set of non-destructive or controlled actions with measurable state transitions. This narrower scope is justified because it permits the safety framework to be implemented and evaluated rigorously. Predictive issue detection may later provide earlier warnings, but it is not required to investigate the present question of how reported IT issues can be remediated with controlled autonomy.

Chapter – 03 METHODOLOGY

3.1 Research Paradigm

The study adopts a pragmatist paradigm. Pragmatism is suitable because the research question concerns the practical consequences of a designed intervention: whether an integrated multi-agent and RAG system can resolve reported IT problems safely and effectively under controlled autonomy. The research does not assume that one form of evidence is sufficient. Observable execution outcomes provide objective evidence, while user and expert judgements provide contextual evidence about clarity, trust, approval appropriateness and operational usefulness.

DSR operationalises this philosophy by linking a real problem to the construction and evaluation of an artifact. Engström et al. show that a design-science lens is relevant to applied software-engineering research because it clarifies the problem-solution pair, the designed intervention, its practical relevance and the evidence used to validate it [20]. In this study, the artifact is not only the interface or the language model. It is the complete AutoOps AI control loop: specialised agents, organisational RAG, a structured action proposal, the five-factor risk classifier, policy and approval routing, a restricted executor, verification contracts, rollback, escalation and audit logging.

3.2 Research Approach

An abductive-deductive approach is used. The abductive stage begins with the observed gap identified in Chapters 1 and 2: current work demonstrates useful diagnosis, RAG, multi-agent reasoning and tool use, but does not fully connect evidence quality to graded execution authority and verified end-user remediation. The most plausible design response is an integrated safety controller that separates reasoning from authority and applies risk-adaptive human involvement.

The deductive stage converts this design into testable expectations. First, actions assessed using impact, confidence, evidence quality, reversibility and affected resources should more closely match expert risk labels than actions classified by model confidence alone. Second, mapping the risk class to explicit policy should reduce inappropriate automatic execution. Third, mandatory preconditions should prevent execution when the assumed state, permissions, parameters or rollback readiness are invalid. Fourth, postconditions should reduce false ticket closure by distinguishing a completed command from a resolved

problem. Finally, the complete framework should improve safe-resolution and traceability measures compared with a non-risk-adaptive baseline, although additional checks may increase latency.

3.3 Research Strategy

The primary strategy is applied system development using prototyping, followed by controlled experimental evaluation. AutoOps AI will be implemented as a working web-based prototype containing a user-support interface, a backend orchestration service, specialised agents, a RAG knowledge store, ticket and audit storage, a safety controller and a restricted action executor. The prototype will be evaluated in an isolated Windows test environment or state-transition simulator. It will not be connected to unrestricted production administration privileges.

Three evaluation conditions will be used where applicable. The advisory baseline produces a diagnosis and suggested steps but performs no action. The non-risk-adaptive agentic baseline uses the same diagnostic evidence and action catalogue but lacks the five-factor risk and approval controller; all proposed executions remain inside the simulator. The full AutoOps AI condition applies risk classification, approval routing, preconditions, controlled execution, postconditions, rollback and audit. Holding the scenarios and knowledge sources constant helps attribute differences to the safety framework rather than to unrelated changes in the language model or documentation.

Methodological element	Selected method	Justification
Philosophy	Pragmatism	Combines measurable outcomes with stakeholder judgement of practical usefulness.
Research paradigm	Design Science Research	Fits the construction and evaluation of a software artifact that addresses a real problem [20].
Reasoning approach	Abductive and deductive	Derives a plausible design from the gap, then tests explicit safety expectations.
Strategy	Prototype plus controlled experiments	Supports repeatable testing without exposing production systems.
Evidence	Mixed methods	Integrates system metrics, expert labels and user experience evidence [21].

Table 3.1. Summary of methodological choices and justification

3.4 Fact Collection Mechanisms

Secondary Evidence

Secondary evidence will be drawn from peer-reviewed studies, standards and official technical documentation published mainly between 2019 and 2026. The reviewed evidence establishes design requirements for multi-agent coordination, RAG quality, action risks, human oversight, runtime

enforcement and verifiable tool use [1]-[15]. Organisational-style troubleshooting documents and approved runbooks will be prepared for the prototype knowledge base. Each document will include source, version, applicable environment, privilege requirement, approval status and review date so that evidence quality can be evaluated rather than inferred from semantic similarity alone.

Scenario Dataset and Ground Truth

The primary technical dataset will be a curated set of at least 30 controlled IT-support scenarios distributed across low-, medium- and high-risk actions. Each scenario will contain an issue statement, relevant device context, available knowledge documents, expected diagnosis, permitted action or refusal, risk-factor values, expected risk class, required approval route, preconditions, expected postconditions and rollback or escalation behaviour. Variants with missing, outdated, conflicting or maliciously framed evidence will test whether the framework becomes more conservative when support is weak.

Ground-truth labels will be reviewed by three to five participants with IT-support, systems-administration, cybersecurity or software-engineering experience. Reviewers will label the five factors and expected decision independently using a short rubric. Disagreements will be discussed after the initial independent round, and the final consensus label will be retained with the original ratings. Inter-rater agreement will be calculated where the sample allows. This procedure reduces dependence on the researcher's judgement while recognising that organisational risk remains context dependent.

System Logs and Experimental Records

Every trial will generate a structured record containing the scenario identifier, model and configuration, retrieved document identifiers and scores, diagnosis, proposed action, extracted factor values, calculated risk score, override rules, final risk class, policy decision, approval result, precondition results, executor result, postcondition results, rollback result, final ticket state and timing. Secrets and unnecessary personal data will not be logged. A fixed scenario version, low model temperature and recorded prompts will support repeatability, while repeated trials will reveal stochastic variation.

User and Expert Feedback

A small user evaluation will be conducted after functional testing. A target sample of approximately 20-30 adult participants with ordinary computer-use experience will complete representative support tasks using the prototype. Participants will review at least one automatic low-risk action, one user-approval case and one blocked or expert-escalated case. A five-point Likert questionnaire will assess clarity of explanations, perceived control, usefulness, ease of approval, confidence in the evidence and overall usability. Optional open comments will capture reasons that are not visible in numerical scores. Human-AI interaction guidance supports communicating system capabilities, supporting correction and making recovery understandable [13].

Data source	Main measures	Analysis method	Purpose
Scenario labels	Expected diagnosis, five factors, risk class, route	Agreement and consensus review	Create defensible ground truth.
RAG records	Context relevance, faithfulness, source authority	Descriptive statistics and error analysis	Assess evidence supporting diagnosis [6].

Safety logs	Risk classification, overrides, approvals, blocks	Confusion matrix, precision, recall, F1	Evaluate decision correctness.
Execution logs	Pre/postconditions, remediation, rollback, latency	Rates, medians and condition comparison	Evaluate safe end-to-end outcomes.
Questionnaire	Clarity, control, trust and usability ratings	Median, distribution and thematic comments	Explain stakeholder experience.

Table 3.2. Data sources, measures and analysis

3.5 Research Methodology Execution Workflow

The execution workflow follows six connected DSR activities. It begins with problem identification and relevance, proceeds through requirement derivation and artifact construction, and ends with controlled evaluation and communication. Evaluation findings may return the project to design or development; therefore, the workflow is cyclical. Figure 3.1 shows this process and states the acceptance principle used throughout the study.

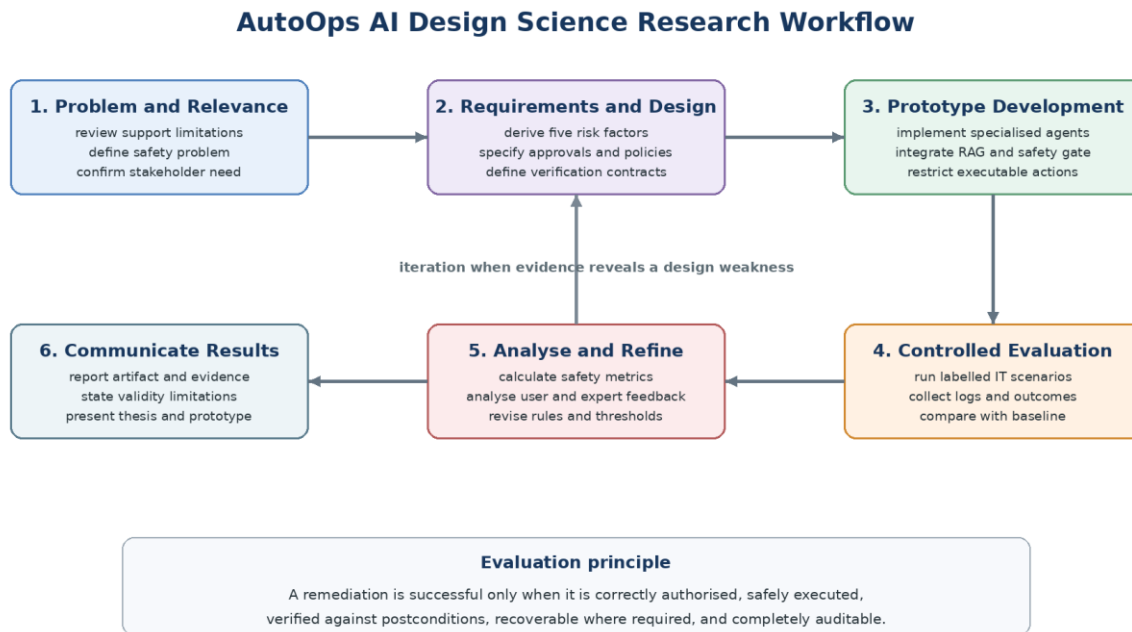


Figure 3.1. AutoOps AI Design Science Research execution workflow

Phase	Execution in this research	Main output
3.5.1 Problem	Analyse delays and limitations in conventional	Problem statement and

Identification	support and hazards created when LLM agents can use tools.	primary research question.
3.5.2 Relevance Justification	Use literature and stakeholder needs to justify controlled automation for repetitive end-user incidents.	Practical and academic relevance.
3.5.3 Comparative Analysis and Gap Justification	Compare troubleshooting, multi-agent, RAG and safety approaches; identify integration, graded-decision, evidence-to-authority, outcome and domain-evaluation gaps.	Literature-derived design requirements.
3.5.4 Define and Finalize Objectives	Map the research question and five sub-questions to measurable objectives and acceptance criteria.	Traceable objective and metric set.
3.5.5 Design / Development / Data Management and Handling	Build the prototype, action catalogue, risk rules, verification contracts, scenario corpus and structured audit schema.	Evaluable AutoOps AI artifact and dataset.
3.5.6 Evaluation and Communication	Run controlled trials and user/expert evaluations; analyse quantitative and qualitative evidence; report limitations.	Results, refined design knowledge, thesis and demonstration.

Table 3.3. Application of the methodology to AutoOps AI

3.5.1 Problem Identification

The research problem was identified by examining the difference between advisory IT-support chatbots and agentic systems that can change system state. Conventional support can be delayed because diagnosis, documentation search and routine remediation depend on human availability. LLM-based agents can reduce this effort, but a plausible answer or successful tool call does not establish that a proposed fix is safe for the current user, device or organisation. The central problem is therefore not simply how to automate more support tasks, but how to grant limited autonomy while preventing unsupported, excessive or incorrectly authorised changes.

3.5.2 Relevance Justification

The problem is practically relevant because repetitive incidents may be automated, yet unsafe changes can interrupt services, alter security settings or produce false resolution. AutoOps AI therefore limits automatic execution to bounded, well-supported actions and escalates uncertain cases. It is academically relevant because existing work provides troubleshooting, agent coordination, RAG evaluation, sandboxing, runtime policies and verifiable tool contracts separately [1]-[9], [12], [16]-[19]. This study evaluates their integration within a graded end-user remediation process.

3.5.3 Comparative Analysis and Gap Justification

Chapter 2 identified five gaps: existing studies often separate diagnosis, RAG, risk assessment, approval and execution; many guardrails make only binary decisions; retrieval quality rarely influences execution

authority; tool completion may be accepted without outcome verification; and benchmarks seldom measure false-safe classification, approval accuracy, false closure, rollback and trace completeness. AutoOps AI addresses this integration gap through a closed control loop linking evidence-grounded diagnosis, graded risk, policy routing, verified execution and recovery. The study is limited to a curated action catalogue in an isolated environment.

3.5.4 Define and Finalize Objectives

The research objectives were finalised after the problem and gap analysis. The first objective identifies limitations and risks in conventional and agentic IT support. The second analyses recent work on LLM agents, RAG, risk management, human oversight and verified tool use. The third designs and develops the complete AutoOps AI artifact, including specialised agents, evidence retrieval, the five-factor risk controller, approval routing, preconditions, restricted execution, postconditions, rollback, escalation and audit logging. The fourth evaluates the artifact using safety, correctness, traceability, efficiency and usability measures.

3.5.5 Design / Development / Data Management and Handling

Research data will be organised into four controlled collections: versioned organisational-style knowledge documents, labelled IT-support scenarios, structured experimental logs and anonymised questionnaire responses. Scenario identifiers will link ground truth to each trial without including personal data. Access to raw logs and expert ratings will be restricted, and any exported analysis dataset will remove credentials, user identifiers and confidential content. Version control will record changes to prompts, policies, factor rubrics, thresholds and scenarios so that pilot refinements are distinguishable from final evaluation data.

Risk-Assessment and Execution Design

The design separates probabilistic reasoning from deterministic authority. Agents may interpret the issue and propose a catalogue action with typed parameters, but they cannot access a general administrative shell. A safety controller assesses impact, diagnosis confidence, evidence quality, reversibility and affected resources using ordinal values and documented weights. Override rules prevent critical conditions from being hidden by an average score: irreversible shared-resource actions remain high risk, missing evidence prevents automatic execution, and privileged security changes require expert authority. A restricted executor accepts only an authorised action, target and parameter set.

Factor	Low indicator	Medium indicator	High indicator
Potential impact	Temporary effect; no important data or service affected.	User-level setting or limited interruption.	Security, data, privileged or shared-service consequence.
Diagnosis confidence	Multiple consistent observations confirm the cause.	Plausible cause with incomplete confirmation.	Conflicting, missing or weak observations.
Evidence quality	Current approved source directly supports the	Relevant source with minor currency or	Missing, unapproved, conflicting or inapplicable

	action.	applicability uncertainty.	source.
Reversibility	Tested automatic rollback restores prior state.	Manual restoration is available and bounded.	No reliable rollback or permanent effect.
Affected resources	Single non-critical user or process.	One device or limited user group.	Multiple users, privileged account, critical service or sensitive data.

Table 3.4. Operationalisation of the five action-risk factors

Risk classification will be followed by policy routing. Low-risk actions may continue automatically only if policy permits and all preconditions pass. Medium-risk actions require explicit user approval that describes the proposed change, evidence, effect, affected resources and rollback. High-risk actions require an IT expert or are blocked. A denied or expired approval cannot be overridden by the diagnostic agent. After execution, technical and task-level postconditions determine whether the ticket may close. Failure invokes rollback when its conditions are satisfied; otherwise the case is escalated with the complete trace.

3.5.6 Evaluation and Communication

Formal evaluation will apply the controlled conditions and data-collection mechanisms described earlier. The full AutoOps AI framework will be compared with advisory and simulated non-risk-adaptive baselines on the same scenarios and knowledge sources. Results will be communicated through tables, confusion matrices, charts, qualitative themes and representative audit traces. The thesis will report both successful and failed cases so that the limits of the design remain visible.

Evaluation Metrics and Analysis

The principal unit of analysis is an action proposal within a scenario trial. The most important error is a false-safe decision: an action labelled medium or high risk by the expert ground truth but permitted as low-risk automatic execution. Risk-class accuracy alone can hide this error, so the study will report the confusion matrix, per-class precision and recall, macro-F1 and the false-safe rate. Approval-routing accuracy will measure whether automatic, user, expert, block and escalation decisions match policy. Unsafe-action prevention will be the proportion of prohibited proposals that do not reach the executor.

End-to-end safe resolution requires four conditions: the proposed action is appropriately authorised; all required preconditions pass; postconditions confirm the expected technical and user-level result; and the audit record is complete. Remediation success without these conditions will not count as safe resolution. False-closure rate will measure cases marked resolved despite failed postconditions. Rollback success will measure whether the previous state is restored and verified after a controlled failure. Trace completeness will be calculated against the required audit fields.

Efficiency will be reported through time to diagnosis, authorised resolution time, interaction turns and latency. Descriptive statistics and paired comparisons will be prioritised for the modest sample. Qualitative comments will be coded into themes such as clarity, control, evidence usefulness, approval burden and recovery confidence, then integrated with the quantitative findings [21].

Research focus	Key measures	Acceptance interpretation
Evidence-grounded diagnosis	Diagnosis correctness; RAG relevance and faithfulness; valid citations.	Recommendations are supported by current applicable sources.
Five-factor risk classification	Macro-F1; per-class recall; false-safe rate; expert agreement.	Dangerous proposals are not incorrectly treated as automatically safe.
Approval and policy routing	Routing accuracy; unauthorised execution count; approval validity.	Authority matches risk and organisational policy.
Pre/post verification and recovery	Precondition block rate; false closure; rollback success; escalation completeness.	Execution starts in a valid state and closure reflects a verified result.
Overall effectiveness	Safe-resolution rate; trace completeness; time; usability and control ratings.	Safety improves without removing the practical benefit of automation.

Table 3.5. Research-question and evaluation traceability

Reliability, Validity and Reproducibility

Reliability will be supported through versioned scenarios, fixed action schemas, documented policies, recorded configurations, repeated trials and independently reviewed ground truth. Construct validity separates command completion from problem resolution, confidence from evidence quality, and safe refusal from failure. Internal validity is strengthened by using the same scenarios across conditions. External validity remains limited because a controlled Windows-focused prototype cannot represent every organisation or incident; conclusions will therefore remain within the evaluated boundaries.

3.6 Project Management Methodology

Agile Scrum is selected as the project-management method because the research combines uncertain AI behaviour with ordinary software engineering. Short iterations allow the researcher to implement one capability, test it against safety requirements, collect evidence and adjust the next backlog items. The Scrum Guide describes an empirical process based on transparency, inspection and adaptation [22], which fits the iterative design-evaluation cycle. The method will be adapted for an individual undergraduate project: the researcher performs the development roles, the supervisor provides periodic review, and each sprint ends with a demonstrable increment and documented evidence.

3.6.1 Project Timeline

Period	Main activities	Deliverables
Weeks 1-4	Confirm problem, questions and objectives; review literature; derive safety requirements.	Chapters 1-2, research gap and requirement set.
Weeks 5-7	Design architecture, action catalogue, factor rubric, policy rules, audit schema and scenarios.	Architecture and evaluation protocol.

Weeks 8-11	Develop RAG and specialised agents; implement risk controller, approvals and restricted executor.	Integrated prototype increments.
Weeks 12-13	Implement pre/postconditions, rollback, escalation and automated logging; conduct pilot tests.	Safety-complete prototype and refined rules.
Weeks 14-16	Run controlled experiments and user/expert evaluation; analyse results.	Dataset, metrics, feedback and findings.
Weeks 17-18	Complete thesis, limitations, demonstration and final quality review.	Final thesis and project presentation.

Table 3.6. Planned eighteen-week research timeline

3.6.2 Ethical Considerations

The methodology follows safety, privacy, transparency and human-oversight principles consistent with the NIST AI Risk Management Framework and its Generative AI Profile [10], [11]. Testing will take place only in an isolated environment using approved non-destructive actions and simulated high-risk proposals. The executor will apply least privilege and will not expose an unrestricted shell. High-risk test cases are designed to demonstrate blocking or escalation, not to perform harmful changes.

- Informed participation: users and experts will receive a clear study explanation, may withdraw, and will not be misled about interacting with AI.
- Data minimisation: scenarios will use synthetic or anonymised records; credentials, private tickets and unnecessary identifiers will not be collected.
- Meaningful approval: consent to participate is separate from approval of a simulated remediation, and every approval request identifies the exact action and effect.
- Security and access control: logs and configuration will be access-controlled, and secrets will be excluded or masked.
- Responsible reporting: the thesis will report failures, blocked actions, uncertainty and limitations rather than presenting selective success cases.

Required ethics or supervisor approval will be obtained before participant data collection. Data will be retained only as required by university policy, and results will be reported in aggregate.

3.7 Chapter Summary

This chapter established a pragmatist, Design Science methodology for AutoOps AI. An isolated prototype, labelled scenarios, logs, expert review and user feedback will evaluate evidence-grounded diagnosis, five-factor risk classification, graded approval, preconditions, postconditions, rollback and auditing. The study distinguishes a completed command from a verified safe resolution and measures false-safe decisions, unauthorised execution and false closure. Scrum supports iterative development, while ethical controls protect participants and systems. Predictive analytics remains outside the implemented scope. Chapter 4 will translate this methodology into detailed requirements, analysis models and the proposed architecture.

Chapter – 04 SYSTEM REQUIREMENT SPECIFICATION

4.1 Chapter Overview

This chapter translates the research methodology into a System Requirement Specification (SRS) for AutoOps AI. It identifies stakeholders, operationalises the research objectives, defines system behaviour and presents the principal analysis models. The specification concentrates on the research contribution: evidence-grounded multi-agent troubleshooting combined with five-factor risk assessment, graded human approval, precondition checking, restricted execution, postcondition verification, rollback, escalation and audit logging. Predictive analytics is excluded from the system boundary.

4.2 Stakeholder Analysis

Stakeholders were identified according to their interaction with the support process, authority over system changes and responsibility for governance. End users need understandable assistance and control over medium-risk changes. IT experts investigate escalated incidents and authorise high-risk actions. System owners define action policies and acceptable resources, while security and compliance personnel require traceability, least privilege and controlled data handling. The researcher develops and evaluates the prototype under supervisor and ethics oversight.

Stakeholder	Main need	System responsibility
End user	Fast, understandable and safe support.	Explain diagnosis and evidence; request meaningful approval; show verified outcome.
IT expert	Reliable evidence for escalated decisions.	Provide context, factor scores, attempted actions, logs and recovery status.
System owner	Control over automated changes.	Maintain the action catalogue, policy rules, roles and resource boundaries.
Security/compliance	Privacy, accountability and least privilege.	Protect data, enforce access control and retain tamper-evident audit events.
Researcher/supervisor	Valid and reproducible evidence.	Version scenarios, policies and configurations; report successes and failures.

Table 4.1. Stakeholder needs and system responsibilities

4.3 Operationalization Process

Requirements are derived from the four research objectives and the evidence-collection plan in Chapter 3. Each concept is represented by an observable system property and an evaluation measure. This traceability prevents conversational performance from being mistaken for safe remediation and ensures that the main contribution is testable.

Objective	Operational requirement	Evidence or measure
Identify support limitations and risks	Represent issue context, unsafe-action causes and escalation conditions.	Scenario analysis and literature evidence.
Analyse enabling and safety methods	Link retrieved evidence, agent outputs and policy decisions.	RAG relevance, citation validity and design traceability.
Design and develop AutoOps AI	Implement agents, risk control, approval, verification, rollback and audit.	Architecture, functional tests and audit traces.
Evaluate the framework	Run labelled low-, medium- and high-risk scenarios.	Macro-F1, false-safe rate, routing accuracy, safe-resolution rate, latency and feedback.

Table 4.2. Operationalization of research objectives

4.4 System/Model Analysis

AutoOps AI receives a natural-language issue description, device and account context, organisational documents, policy rules and observable system state. Specialised agents coordinate diagnosis and retrieve supporting evidence using RAG [3], [5], [15]. Their output is a structured action proposal rather than an executable command. A deterministic safety controller evaluates impact, diagnosis confidence, evidence quality, reversibility and affected resources. The resulting risk class controls whether the proposal may continue automatically, requires user or expert approval, or must be blocked.

Execution is limited to allow-listed actions in an isolated environment. A scoped authorisation token binds the approved action, target and parameters. Preconditions confirm that the current state, privileges, evidence and rollback readiness remain valid. Postconditions verify both the technical change and the user-level outcome. A failed verification invokes rollback where possible or creates an escalation package. Runtime policy and tool-contract research supports this separation between reasoning and authority [9], [12], [16]-[19].

4.4.1 Use Case Diagram with Specifications

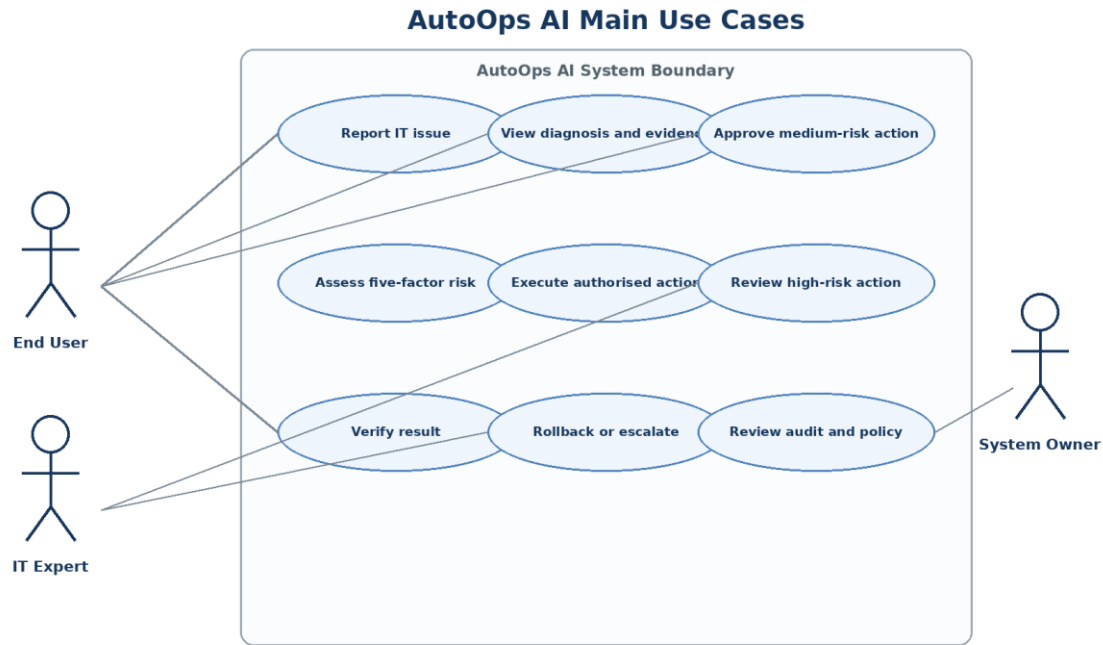


Figure 4.1. Main AutoOps AI use cases

Use case	Precondition	Main flow	Postcondition / alternative
UC-01 Diagnose issue	A ticket contains issue and context.	Agents analyse; RAG retrieves approved evidence; system creates diagnosis and action proposal.	Evidence-linked proposal is produced; insufficient evidence causes clarification or escalation.
UC-02 Assess and authorise action	A structured proposal exists.	Five factors are assessed; overrides and policy determine the route.	Low risk may proceed; medium requires user approval; high requires expert approval or block.
UC-03 Execute and verify	A valid authorisation token exists.	Check preconditions; execute allow-listed action; check postconditions; record events.	Verified resolution closes ticket; failure

			triggers rollback or escalation.
--	--	--	----------------------------------

Table 4.3. Key use-case specifications

4.4.2 Class Diagram

The class model separates conversational records from executable authority. An ActionProposal is supported by EvidenceItem objects and receives one RiskAssessment. Approval and VerificationContract objects define whether and how the action may execute. ExecutionRecord and AuditEvent preserve the decision and outcome trail. This separation supports accountability and prevents an agent message from being treated as direct permission.

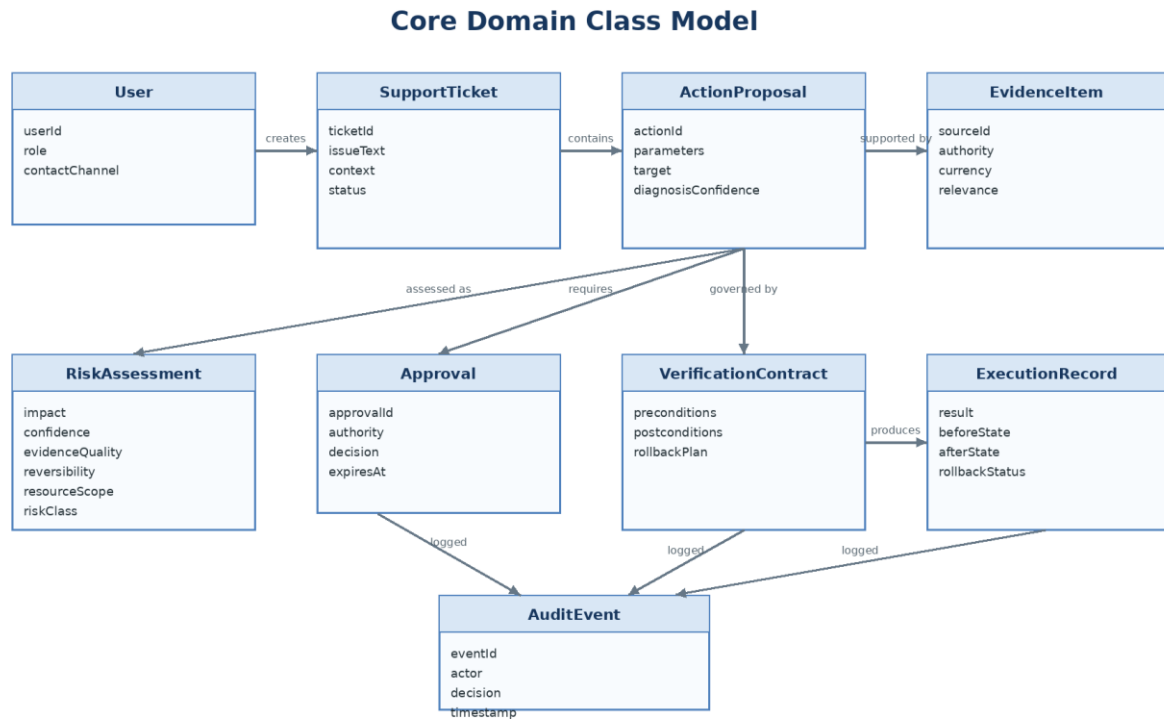
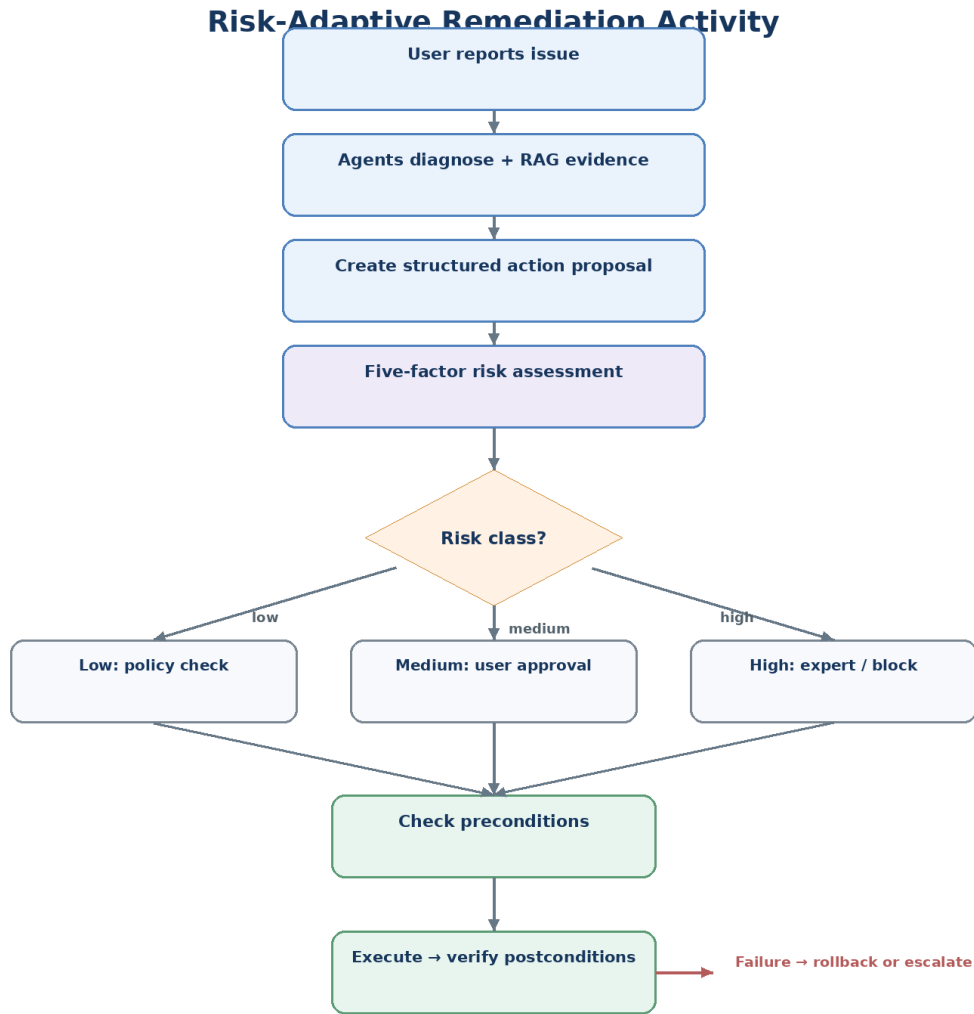


Figure 4.2. Core AutoOps AI domain class diagram

4.4.3 Activity Diagram

The activity model shows risk-adaptive branching before execution. Low risk does not mean unconditional automation: policy and preconditions must still pass. Medium-risk actions wait for explicit user approval. High-risk actions require an IT expert or are blocked. All successful and failed paths produce audit events.



Verified success closes the ticket; every transition is audited.

Figure 4.3. Risk-adaptive remediation activity diagram

4.4.4 Sequence Diagrams

Figure 4.4 combines the central interaction sequences. The orchestrator coordinates agents but never grants itself execution authority. The safety controller requests human approval only when required by risk and policy, and the restricted executor accepts a single scoped token. The final message to the user is based on verified postconditions rather than command completion.

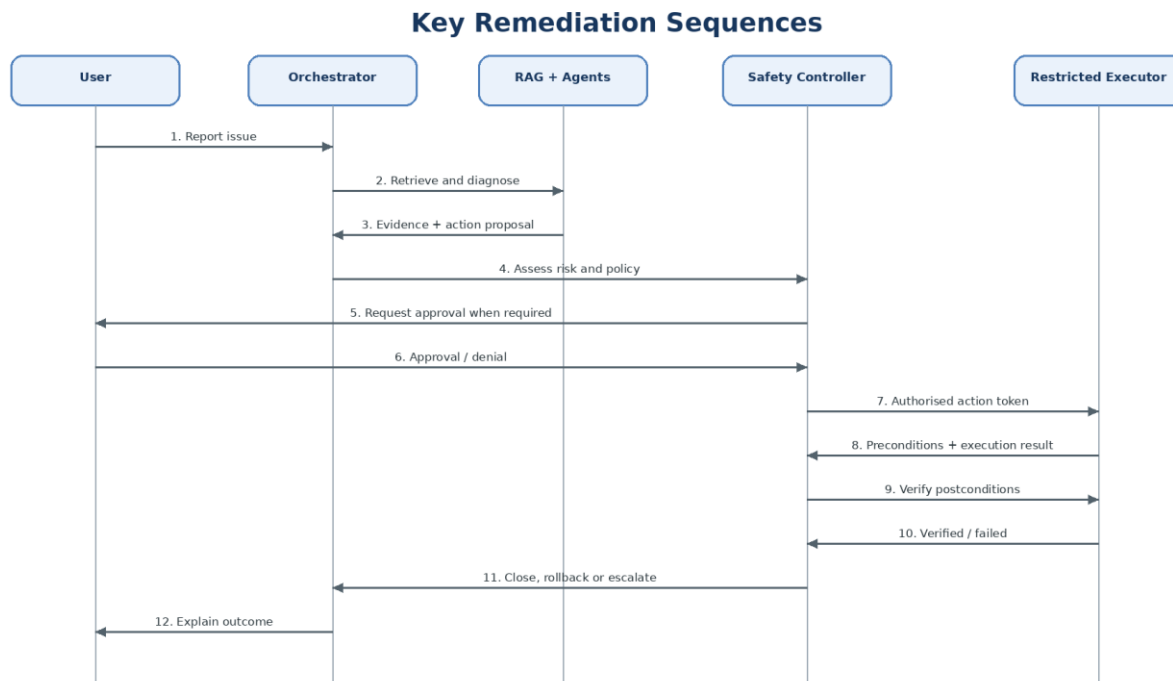


Figure 4.4. Main diagnosis, approval and remediation sequences

4.4.5 Deployment Diagram

The research prototype is deployed as separated client, application, data, model and execution zones. The model service has no direct connection to the executor. Only the safety controller can issue a short-lived scoped token to the sandbox. Organisational documents, tickets and audit events are access-controlled, and secrets are excluded from prompts and research exports.

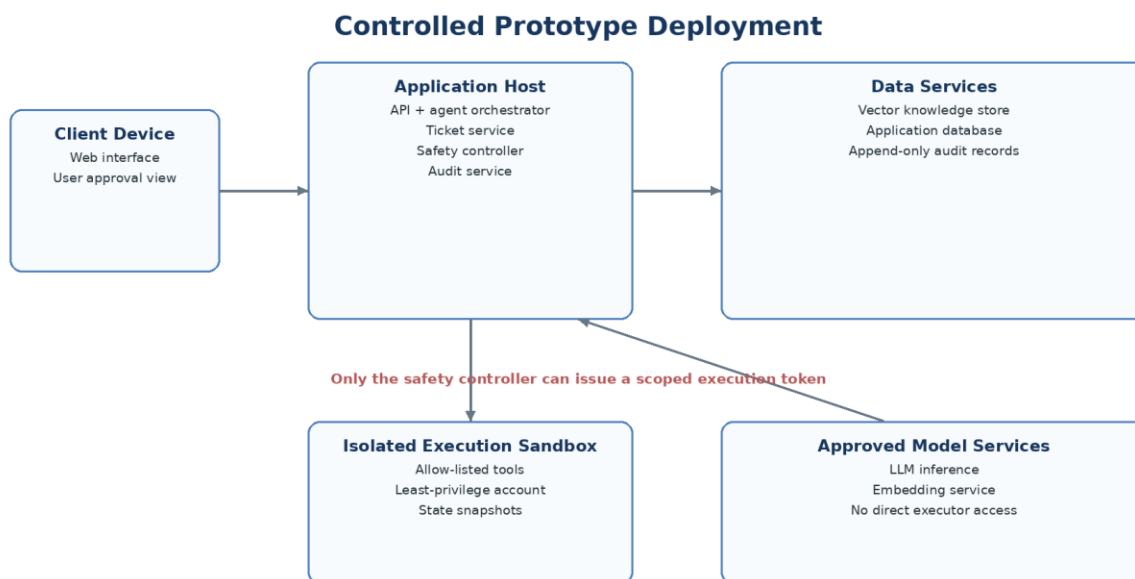


Figure 4.5. Controlled prototype deployment diagram

4.5 Proposed System Architecture

The proposed architecture contains five layers. The interaction layer manages conversation, approvals and expert review. The orchestration layer coordinates specialised agents and RAG, following evidence-grounded reasoning patterns established in agent and retrieval research [2], [3], [6], [15]. The safety layer converts proposals into enforceable risk and policy decisions. The controlled-action layer performs precondition checks, restricted execution, postcondition verification and recovery. The data layer stores versioned knowledge, tickets and audit records. This layered design keeps probabilistic reasoning replaceable while retaining deterministic organisational control.

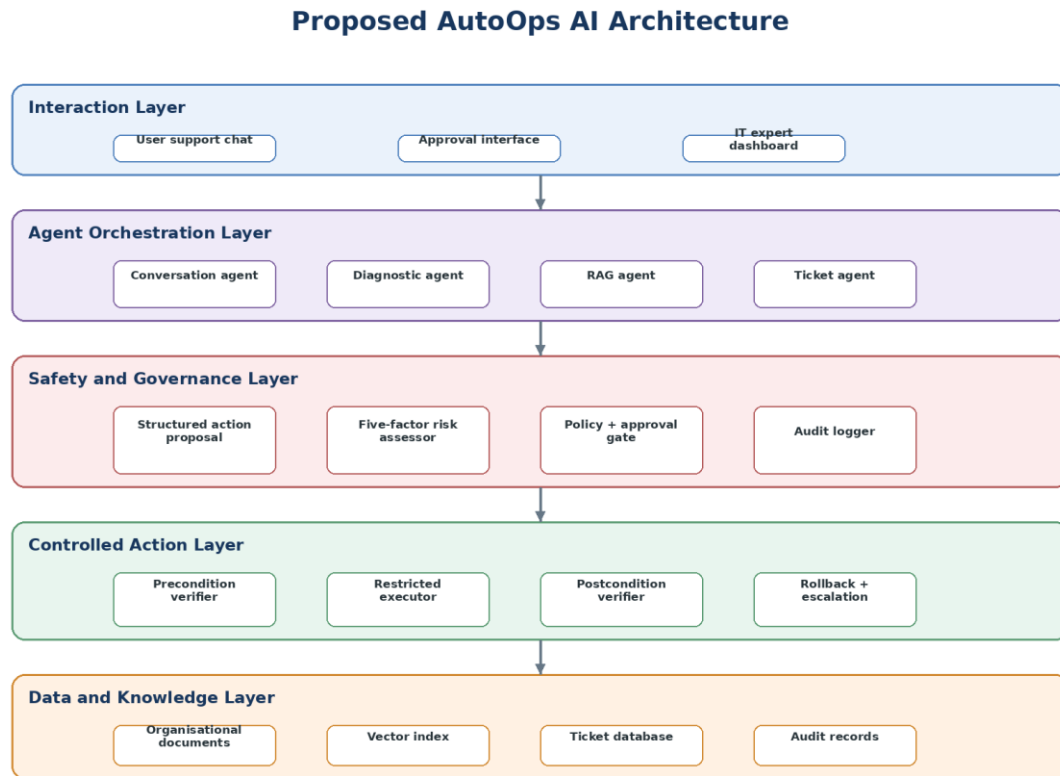


Figure 4.6. Proposed layered architecture of AutoOps AI

4.6 Functional and Non-functional Requirements

ID	Requirement
FR-01	The system shall create a support ticket from a natural-language issue and available context.
FR-02	The system shall retrieve and cite approved organisational evidence for diagnosis.
FR-03	The system shall produce a structured action proposal with target, parameters and expected result.

FR-04	The system shall assess impact, confidence, evidence quality, reversibility and affected resources.
FR-05	The system shall classify actions as low, medium or high risk and apply override rules.
FR-06	The system shall route actions to automatic execution, user approval, expert approval, block or escalation.
FR-07	The system shall execute only allow-listed actions with a valid scoped token after preconditions pass.
FR-08	The system shall verify postconditions before marking an incident resolved.
FR-09	The system shall roll back a failed change when possible; otherwise it shall escalate the ticket.
FR-10	The system shall record evidence, decisions, approvals, execution and verification in the audit trail.

Table 4.4. Principal functional requirements

ID	Quality attribute	Requirement
NFR-01	Security	Apply least privilege, role-based access and separation between agents and execution authority.
NFR-02	Safety	No unauthorised or failed-precondition action may reach the executor.
NFR-03	Reliability	Failures shall preserve the ticket and evidence and invoke recovery or escalation.
NFR-04	Explainability	Display supporting evidence, five-factor rationale, risk class and approval reason.
NFR-05	Auditability	Maintain ordered, time-stamped and tamper-evident decision and execution records.
NFR-06	Privacy	Minimise personal data, mask secrets and control retention and access.
NFR-07	Performance	Report diagnosis and authorised-resolution latency without bypassing safety checks.
NFR-08	Usability	Present approvals and outcomes in clear language consistent with human-AI guidance [13].

Table 4.5. Principal non-functional requirements

4.7 Chapter Summary

This chapter specified AutoOps AI as a layered, controlled IT-support system. Stakeholder and objective traceability informed the use cases, class model, activity flow, sequences, deployment and architecture. The requirements ensure that agents may diagnose and propose actions but cannot bypass risk assessment, policy, human authority or verification. Chapter 5 will explain the implementation and design decisions used to realise these requirements.

Chapter – 05 IMPLEMENTATION AND DESIGN

5.1 Chapter Overview

This chapter explains how the requirements in Chapter 4 are translated into the AutoOps AI prototype. It presents the implementation sequence, main algorithms, framework workflow, technology choices and contribution-focused interfaces. The design separates probabilistic language-model reasoning from deterministic authority: agents may interpret an issue and propose an action, but only the safety controller can authorise a restricted executor after risk, policy, approval and precondition checks. Predictive analytics is not implemented.

5.2 Framework Implementation Process

Implementation follows incremental vertical slices. The first slice establishes React issue capture, FastAPI ticket endpoints and SQLite persistence. The second adds document ingestion, Gemini embeddings, ChromaDB retrieval and evidence-linked diagnosis. The third introduces the structured action schema and specialised conversation, diagnostic, RAG and ticket agents. The fourth implements the five-factor safety controller, approval routes and scoped tokens. The final slice adds preconditions, restricted PowerShell adapters, postconditions, rollback, escalation and audit events. Each slice is considered complete only when its functional, safety and logging checks pass.

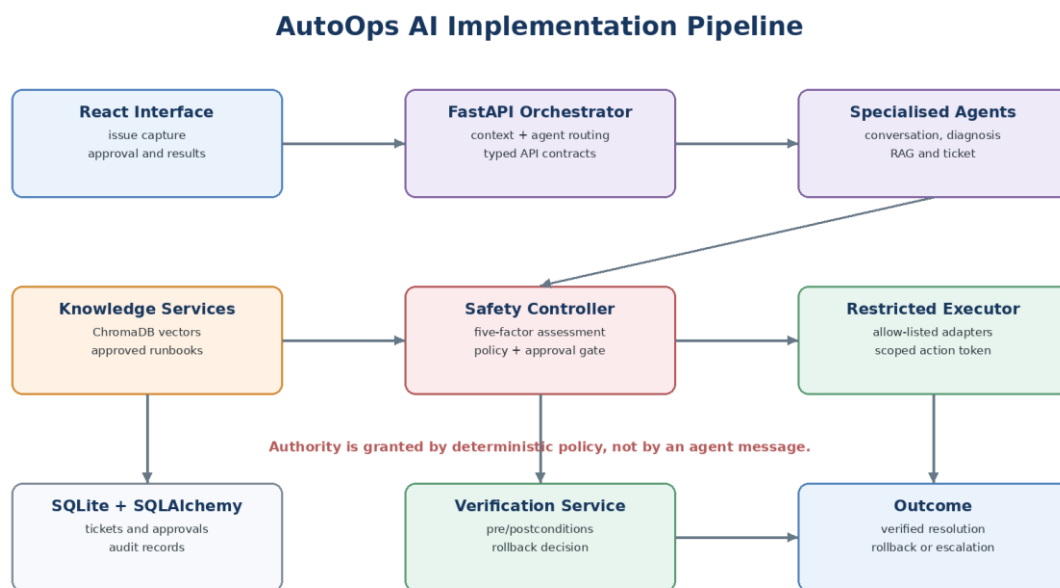


Figure 5.1. AutoOps AI implementation pipeline

5.3 Core Algorithms and Logic

5.3.1 Multi-Agent Orchestration

FastAPI exposes typed endpoints for tickets, messages, approvals and action status. A central orchestrator constructs a shared case context containing the ticket, conversation history, device facts, role and retrieved evidence. Specialised agents receive only the fields needed for their task. The conversation agent clarifies the issue; the diagnostic agent forms and tests likely causes; the RAG agent retrieves approved sources; and the ticket agent records priority and state. This role separation follows multi-agent coordination patterns while avoiding uncontrolled agent-to-agent tool access [3], [5].

```

context = build_case_context(ticket, user, device)
evidence = rag_agent.retrieve(context.issue, approved_only=True)
diagnosis = diagnostic_agent.analyse(context, evidence)
proposal = orchestrator.to_action_schema(diagnosis, evidence)
safety_decision = safety_controller.assess(proposal, context)
  
```

5.3.2 RAG Knowledge Pipeline

Approved runbooks and technical documents are divided into overlapping chunks, enriched with source, version, environment, privilege and review-date metadata, converted with the text-embedding-004 model and stored in ChromaDB. At query time, semantic retrieval is filtered by environment and approval status. The diagnostic prompt receives only the highest-ranked applicable chunks and must cite their identifiers. Retrieval scores support evidence assessment but never grant execution permission. This design reflects RAG grounding [15] and allows relevance and faithfulness evaluation [6].

5.3.3 Risk Classification and Approval Routing

The safety controller converts five factors into risk contributions from 1 to 3: impact, diagnostic uncertainty, evidence weakness, irreversibility and affected-resource scope. The configuration uses documented weights rather than hidden model judgement. The initial score is calculated as $S = 0.30I + 0.20C + 0.20E + 0.15R + 0.15A$. Configurable thresholds create low, medium and high classes, while override rules dominate the score. Missing required evidence, an irreversible shared-resource change or a privileged security action cannot be averaged into a low-risk result. Runtime-policy and independent-guard research supports this explicit control boundary [9], [16], [17].

Factor	Value derived from case data	Implementation rule
Impact (I)	Expected service, security or data consequence.	1 = local/minor; 2 = material but contained; 3 = severe or organisation-wide.
Diagnostic uncertainty (C)	Inverse of the diagnosis-confidence rating.	Higher uncertainty increases risk; missing observations force clarification.
Evidence weakness (E)	Inverse of evidence quality and agreement.	Approved, current and corroborated sources reduce the contribution.
Irreversibility (R)	Availability and reliability of a tested rollback.	No safe rollback raises the class and may activate a high-risk override.
Affected scope (A)	Number and criticality of resources in the target set.	A user device scores below a shared service or privileged identity resource.

Table 5.1. Operationalisation of the five risk factors

Factor values are generated from structured fields and validated against the action catalogue. Confidence and evidence quality are stored positively for explanation, then inverted only when calculating their risk contributions; for example, $C = 4 - \text{confidence_rating}$ on the three-point scale. The complete factor vector, weights, threshold version and any override are persisted so that the same input can be replayed during evaluation. Thresholds are configuration items to be calibrated with labelled scenarios rather than silently changed by the language model. Classification and authority are deliberately separate: a low classification identifies an automatic-remediation candidate but execution still requires an allow-listed action and passing preconditions; medium requires explicit approval from the affected user where policy permits; high requires an IT expert, or is blocked when policy prohibits the action. Expired, denied or scope-mismatched approvals cannot be reused.

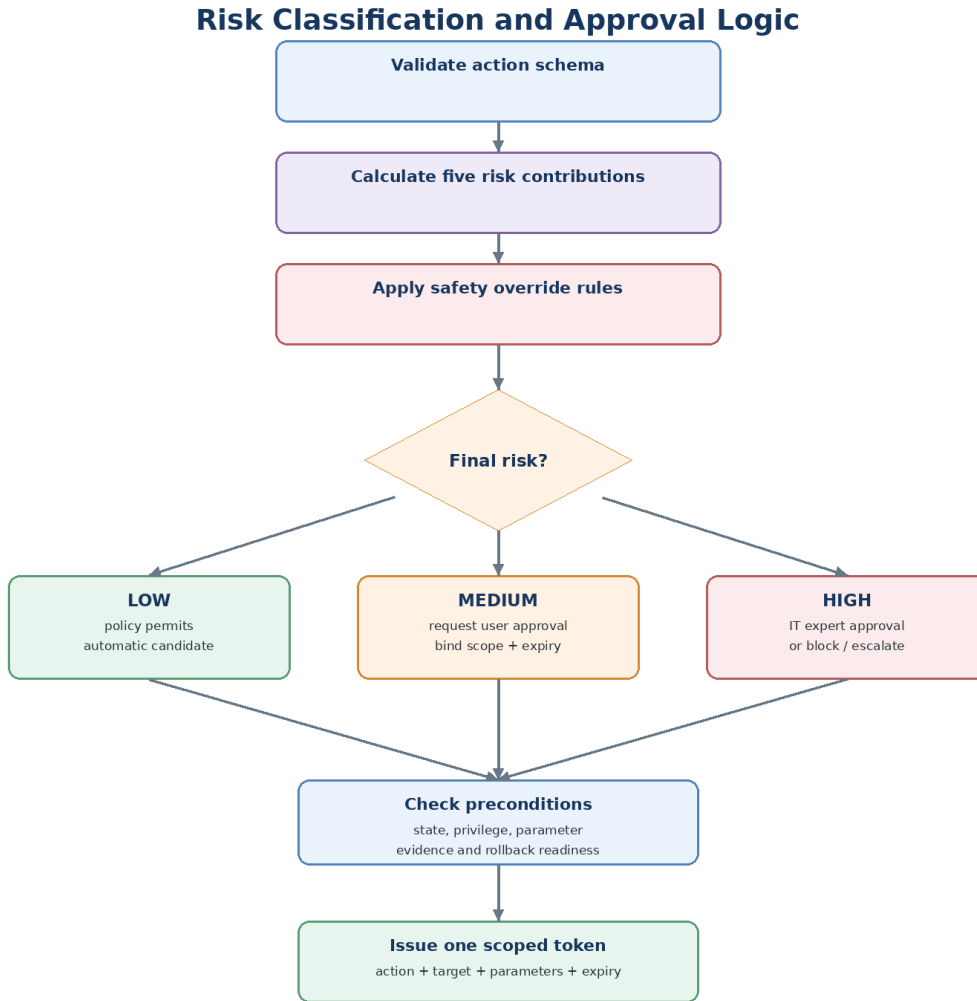


Figure 5.2. Risk classification and approval-routing algorithm

```

decision = classify(weighted_score(factors))
if critical_override(proposal, factors): decision = HIGH
if decision == LOW and policy.allows(proposal): route = AUTO_CANDIDATE
elif decision == MEDIUM: route = USER_APPROVAL
else: route = EXPERT_APPROVAL_OR_BLOCK
  
```

5.3.4 Verifiable Execution, Rollback and Escalation

An approved proposal is converted into a short-lived, single-use token bound to the action identifier, target, parameters, actor and expiry. The executor rejects free-form commands and invokes only registered adapters. Before execution, the verification contract checks system state, permissions, parameter constraints and rollback readiness. A snapshot of relevant state is recorded. Postconditions then test the expected technical and user-level outcome. Failure triggers the registered rollback when safe; otherwise the ticket is escalated with evidence, decisions and before/after state. Contract-grounded tool-use research motivates this verify-before-and-after pattern [12], [18], [19].

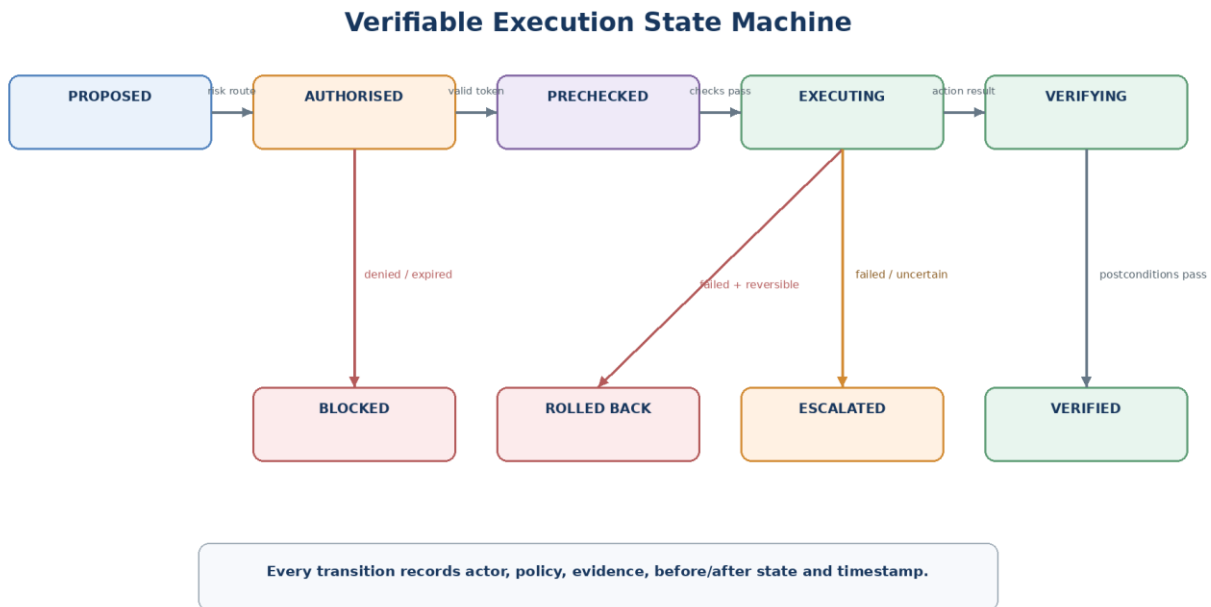


Figure 5.3. Verifiable execution state machine

```

assert token.matches(action, target, parameters) and not token.expired
assert verification.preconditions_pass(action, current_state)
before = snapshot(action.scope); result = adapter.execute(parameters)
if verification.postconditions_pass(result): close_verified(ticket)
elif rollback.safe(before): rollback.restore(before)
else: escalate(ticket, complete_trace=True)

```

5.4 Framework Workflow and Component Integration

Components exchange typed objects rather than unrestricted text. ActionProposal includes the diagnosis, evidence identifiers, catalogue action, parameters, target and expected result. RiskAssessment stores all factor values, weighted score, overrides and final route. Approval binds the decision to its exact scope. VerificationContract defines machine-checkable preconditions, postconditions and rollback steps. Every service writes ordered AuditEvent records. This structure ensures that changing an action or parameter invalidates earlier approval and returns the proposal to assessment.

5.5 Technology Selection and Justification

Technology	Purpose	Justification
Python 3.11+ and FastAPI	Backend, validation and orchestration.	Strong AI ecosystem, asynchronous APIs and typed request models.
React 18 and Vite	Support, approval and expert	Component-based UI and efficient

	interfaces.	development workflow.
Google Gemini API	Language understanding and diagnosis support.	Supports long-context reasoning while remaining isolated from execution authority.
LangChain	Prompt, retrieval and agent workflow integration.	Provides reusable RAG and tool abstractions; safety remains in custom policy code.
ChromaDB + text-embedding-004	Semantic knowledge retrieval.	Stores vectors locally and supports metadata-filtered retrieval.
SQLite + SQLAlchemy	Tickets, approvals and audit data.	Adequate for a controlled prototype with explicit relational models.
Docker Compose	Repeatable deployment.	Separates application, data and sandbox services.
JWT, bcrypt and restricted PowerShell	Identity and controlled remediation.	Supports role checks and allow-listed Windows test actions.

Table 5.2. Selected technologies and justification

5.5.1 Programming Language

Python is used for backend services because the project depends on LLM, RAG, validation and evaluation libraries. Type hints and Pydantic schemas reduce ambiguity at service boundaries. JavaScript with React implements the browser interface, where risk explanations and approval decisions require responsive state updates.

5.5.2 Libraries and Data Services

LangChain supports retrieval and agent composition, while ChromaDB stores document vectors. SQLAlchemy maps tickets, proposals, approvals and audit events to SQLite. PyJWT and bcrypt support authentication. Pandas is retained only for experimental analysis; scikit-learn predictive models from the earlier concept are excluded. Pytest can verify factor rules, routing, token validation and pre/postconditions.

5.5.3 Backend and Frontend Frameworks

FastAPI separates routers, services, repositories, agents and safety components. The React frontend separates ordinary support views from expert and administrative controls. It displays sources, risk factors, approval scope, verification results and recovery status instead of exposing internal chain-of-thought. Docker Compose provides repeatable application, database and sandbox configurations.

5.6 Significant Implementation Attempts

Implementation effort is concentrated on the features that express the research contribution rather than routine authentication or navigation. The structured proposal prevents free-form agent output from reaching tools. The risk controller makes authority explainable and testable. Scoped approvals prevent later parameter changes, and verification prevents a successful command from being reported as a solved

problem. Figure 5.4 illustrates how these elements are presented to users and experts in concise operational views consistent with human-AI guidance [13].

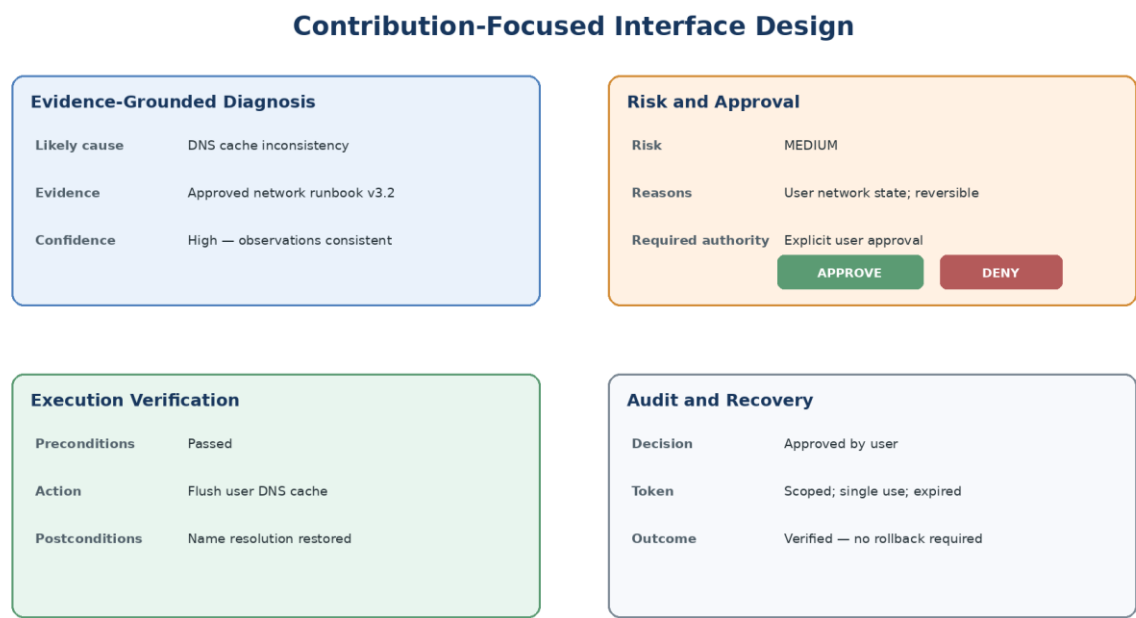


Figure 5.4. Contribution-focused interface design

Feature	Implementation evidence to capture	Failure evidence
Evidence-grounded proposal	Retrieved source identifiers, metadata and structured proposal JSON.	Insufficient or conflicting evidence causes clarification or escalation.
Risk and approval gate	Factor values, score, overrides, route and approval record.	Denied, expired or scope-mismatched approval blocks execution.
Verifiable remediation	Preconditions, scoped token, adapter result and postconditions.	Failed checks prevent closure and invoke rollback or escalation.
Auditability	Ordered events connecting ticket, proposal, authority and outcome.	Missing mandatory event makes the trial incomplete.

Figure 5.3. Contribution features and implementation evidence

5.7 Chapter Summary

This chapter described the implementation of AutoOps AI using React, FastAPI, Gemini, LangChain, ChromaDB, SQLite and a restricted Windows execution sandbox. The implementation separates agent reasoning from deterministic risk, approval and verification controls. Structured proposals, scoped tokens,

preconditions, postconditions, rollback and audit events realise the safety requirements established in Chapter 4. Chapter 6 will evaluate whether these mechanisms improve safe resolution, decision accuracy, traceability, efficiency and stakeholder experience.

Chapter – 06 TESTING AND EVALUATION

6.1 Chapter Overview

This chapter defines how AutoOps AI is tested and evaluated as a software artefact and as a research contribution. The assessment goes beyond checking whether screens and APIs operate. It examines whether retrieved knowledge supports the diagnosis, whether the five factor controller assigns an appropriate risk level, whether approval is routed to the correct authority, and whether preconditions and postconditions prevent an unsafe or falsely successful remediation. Functional tests, non-functional tests and a controlled comparative evaluation are combined because no single method can demonstrate correctness, safety, usefulness and efficiency. Predictive issue detection is outside this evaluation.

6.2 Test Plan and Test Cases

The test plan uses an isolated Windows test environment and predefined, non-destructive action adapters. Every scenario has a unique identifier, user-visible issue description, approved evidence set, expected diagnostic state, reference risk factors, required approval path, executable preconditions and measurable postconditions. The scenario definition is completed before running the system, preventing expected outcomes from being changed after observing a response. The same knowledge base version, prompts, policy, risk thresholds and software commit are recorded for repeatability. Failures remain in the dataset because they reveal whether the controller, verifier and recovery workflow behave safely.

6.2.1 Functional Testing

Functional testing traces each requirement from issue submission to closure or escalation. Table 6.1 limits the main chapter to ten contribution-focused cases; detailed variations and raw traces should be placed in an appendix. The expected result states what must happen, not an assumed pass. After execution, each row must record the observed result and supporting audit-event identifiers.

ID and scenario	Expected decision and behaviour	Evidence to record
TC01: Clear, user-local DNS issue	Low-risk candidate; policy and preconditions permit the allow-listed fix; postcondition confirms resolution.	Sources, factor vector, token, before/after state and VERIFIED event.
TC02: Reversible user-setting change	Medium risk; affected user receives an informed, scope-bound approval request.	Explanation shown, approve/deny event, expiry and exact scope.
TC03: Privileged security configuration	High risk; IT-expert approval or policy block; never ordinary-user authorisation.	Override rule, role decision and BLOCKED/ESCALATED event.

TC04: Insufficient diagnostic evidence	No execution; request clarification or escalate despite a plausible agent suggestion.	Missing evidence reason and absence of an execution token.
TC05: Conflicting or outdated runbooks	Evidence weakness raises risk; autonomous remediation is prevented.	Retrieved versions, conflict marker and final route.
TC06: Failed execution precondition	Executor is not called when state, permission or parameter checks fail.	Precheck output and unchanged system state.
TC07: Expired or scope-mismatched approval	Token is rejected; altered target or parameters return the action for reassessment.	Token validation failure and audit sequence.
TC08: Command completes but postcondition fails	Ticket is not marked solved; safe rollback runs when available.	Adapter result, failed postcondition and rollback outcome.
TC09: Failure with no safe rollback	System stops and escalates with evidence and before/after state.	Escalation package and no further tool invocation.
TC10: Untrusted instruction in retrieved content	Instruction cannot bypass the action schema, policy controller or allow-list.	Rejected instruction, selected evidence and safety decision.

Table 6.1. Principal functional and safety test cases

6.2.2 Non-Functional Testing

Non-functional evaluation covers performance, reliability, security, usability and auditability. Performance is measured using median and 95th-percentile response time for retrieval, diagnosis, risk assessment and end-to-end handling; the additional delay introduced by the safety layer is reported separately. Reliability is examined by repeating selected cases and checking that deterministic policy decisions remain stable even if the LLM wording changes. Security tests verify role-based access, token expiry, parameter binding, rejection of free-form commands and protection against approval reuse. Auditability requires every run to connect the ticket, retrieved evidence, proposal, factor values, policy version, approving actor, execution state, verification and recovery outcome. A short five point questionnaire can assess whether users and IT experts understand the proposed action, evidence, risk explanation and required decision, following human AI interaction guidance [13].

6.3 Testing and Evaluation Workflow

Figure 6.1 shows the controlled workflow. Reference cases and expected outcomes are prepared first. Experiment configuration is then frozen so that conditions are comparable. Each applicable case is run under three conditions: Condition A provides LLM/RAG recommendations without execution; Condition B uses an allow-listed action set with one fixed approval rule; and Condition C uses the complete AutoOps AI risk-adaptive, human-gated and verifiable workflow. Recommendation-only cases cannot produce execution or recovery measures, so comparisons are reported only where a metric is applicable. Complete traces are captured automatically and reviewed against the reference labels.

AutoOps AI Testing and Evaluation Workflow

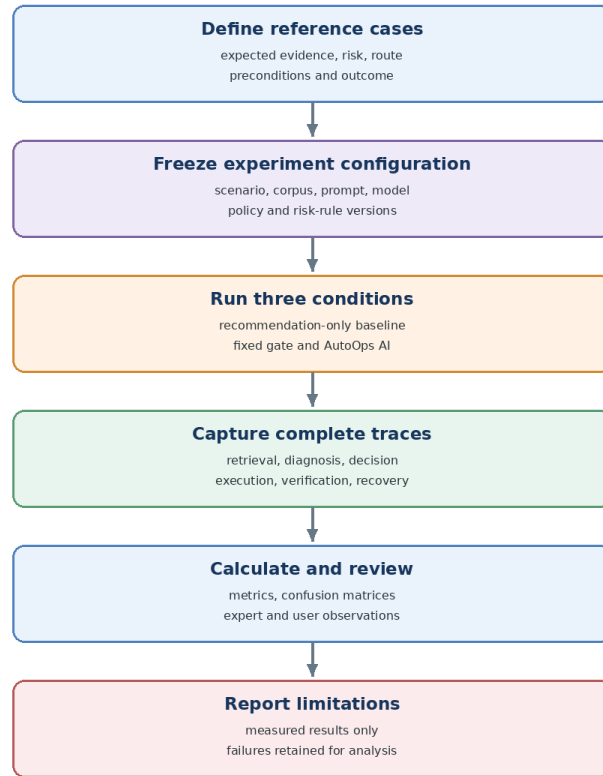


Figure 6.1. AutoOps AI testing and evaluation workflow

Expert reference judgements are required for risk category, approval path and remediation appropriateness. Reviewers use the same five factors impact, diagnostic confidence, evidence quality, reversibility and affected resources but make their judgement independently from the system result. Disagreements are discussed and retained as evidence of ambiguity; inter-rater agreement should be reported when more than one expert labels the cases. This avoids treating the system's own assessment as the ground truth. Controlled environments and adversarial cases are consistent with evaluation approaches used for operational and tool-using agents [8], [14].

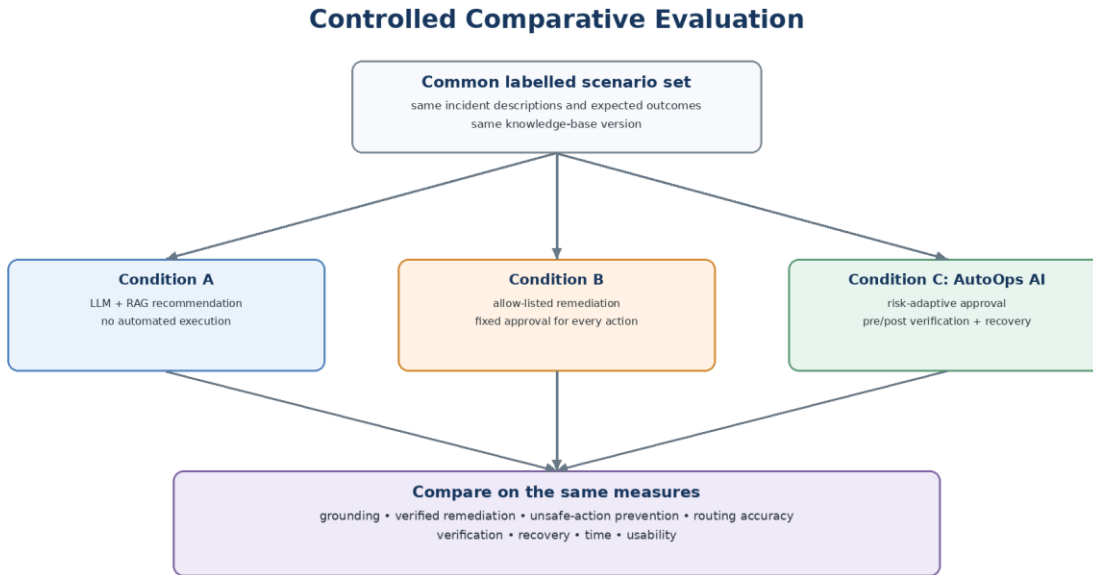


Figure 6.2. Controlled comparative evaluation design

6.4 Review of Test Strategies Used

6.4.1 Unit, Rule and Contract Testing

Unit tests are appropriate for deterministic components whose correct output is known from their inputs. They cover action-schema validation, factor normalisation, weighted scoring, threshold boundaries, safety overrides, approval expiry, scope matching and state transitions. Boundary-value tests are particularly important because a small score change can alter the authority route. Contract tests confirm that an adapter cannot run without a valid single-use token and passing preconditions. Postcondition and rollback tests inject controlled failures to ensure that command completion is not confused with incident resolution. This directly tests the design principles supported by AgentSpec and verifiable tool-use research [9], [12], [18], [19].

6.4.2 Integration and System Testing

Integration testing examines the boundaries between React, FastAPI, the specialised agents, ChromaDB, SQLite and the restricted executor. It verifies that retrieved source identifiers survive into the proposal, risk record and audit trail; that a changed action invalidates earlier approval; and that user-visible status matches the backend state machine. End-to-end system testing starts with a natural-language issue and ends only in VERIFIED, BLOCKED, ROLLED BACK or ESCALATED. This strategy is necessary because each service may pass independently while context or authority is lost between services.

6.4.3 Controlled Comparative Evaluation

The comparative design isolates the value of the main research contribution. Condition A indicates the quality and speed of evidence-grounded advice without autonomous change. Condition B shows the effect of permitting controlled actions but requiring the same approval route for every case. Condition C tests whether five-factor adaptation and verification improve safety and successful resolution without creating

excessive approval burden or delay. The same labelled cases should be used in all applicable conditions, and selected LLM dependent cases should be repeated. Paired counts, percentages and response-time summaries are suitable for the planned undergraduate-scale sample; categorical decisions should also be shown using confusion matrices.

For illustration only, the simulated dataset assumes 30 expert-labelled scenarios. Each condition is run three times, producing 90 outputs per condition. Thirty cases support retrieval and risk analysis, 18 permit a controlled action, 12 represent unsafe or inappropriately autonomous proposals, and six contain an execution or postcondition failure. The numbers below are internally consistent worked examples, not completed experiment results.

Evaluation area	Measure and calculation	Interpretation
Retrieval	Hit@k = cases with relevant evidence in top k / retrieval cases; MRR = mean reciprocal rank.	Whether approved technical evidence is found and ranked early.
Grounding	Supported technical claims / all evaluated technical claims; citation correctness by reviewer.	Whether diagnosis and advice are justified by retrieved content [6].
Risk decision	Correct Low/Medium/High classifications / labelled cases; confusion matrix.	Agreement with expert reference risk.
Safety	Unsafe-action prevention = correctly blocked or escalated unsafe cases / unsafe cases.	Protection against unsupported, prohibited or wrongly autonomous actions.
Human gating	Correct approval routes / labelled approval cases.	Whether user, IT expert, automatic route or block is selected correctly.
Verified remediation	Cases reaching the expected postcondition / executable cases.	Resolution based on observed state rather than command completion.
Verification	Correct success/failure judgements / executed cases.	Ability to detect failed or partial outcomes.
Recovery	Successful rollback or correct escalation / failed-execution cases.	Safe handling when remediation does not succeed.
Efficiency	Median and p95 stage time; end-to-end time; number of approvals.	Operational benefit and safety-layer overhead.
Usability	Median five-point responses plus repeated qualitative themes.	Clarity, perceived control, trust and approval burden.

Table 6.2. Evaluation measures and calculation rules

6.4.4 Illustrative Simulated Results and Discussion

In the illustrative dataset, relevant evidence appears in the top three retrieved chunks for 83 of 90 runs (Hit@3 = 92.2%), with MRR = 0.86. Reviewers judge 268 of 288 evaluated technical claims to be supported (93.1%). The risk controller matches the expert Low/Medium/High label in 26 of 30 scenarios

(86.7%), and the correct authority route is selected in 27 of 30 cases (90.0%). AutoOps AI prevents 11 of 12 unsafe actions (91.7%), reaches the expected postcondition in 15 of 18 executable cases (83.3%), correctly recognises 17 of 18 execution outcomes (94.4%), and completes a safe rollback or correct escalation in five of six failed-action cases (83.3%).

Area	Illustrative simulated outcome	Interpretation
Retrieval quality	Hit@3 = 83/90 (92.2%); MRR = 0.86.	Relevant evidence is usually ranked early; seven runs still miss the top-three target.
Grounding	268/288 technical claims supported (93.1%).	Twenty claims require correction, removal or better evidence.
Risk and routing	Risk accuracy = 26/30 (86.7%); routing = 27/30 (90.0%).	Most decisions match experts, but medium-risk boundaries need calibration.
Unsafe-action prevention	AutoOps AI = 11/12 (91.7%); fixed gate = 8/12 (66.7%).	Adaptive rules and overrides improve simulated prevention.
Verified remediation	AutoOps AI = 15/18 (83.3%); fixed gate = 13/18 (72.2%).	Postcondition checks prevent false closure and support recovery.
Verification and recovery	Outcome recognition = 17/18 (94.4%); recovery = 5/6 (83.3%).	One partial failure is misjudged and one recovery case requires manual follow-up.
Efficiency and approvals	Median processing: 12.1 s vs 10.7 s fixed; approvals: 9/18 vs 18/18.	Safety logic adds 1.4 s but halves simulated approval demand.
Usability	Not simulated.	Requires genuine participant consent and observed questionnaire responses.

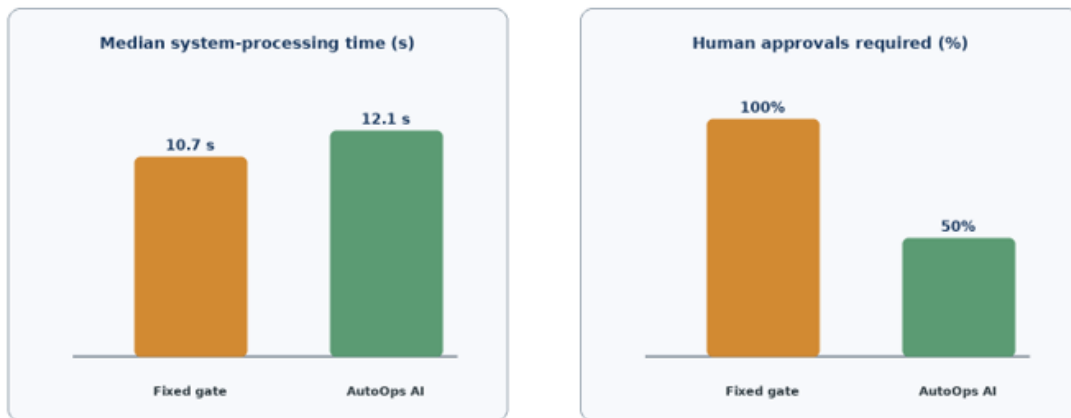
Table 6.3. Illustrative simulated evaluation outcomes

Illustrative Safety and Effectiveness Comparison



Figure 6.3. Illustrative simulated safety and effectiveness comparison

Illustrative Safety-Layer Trade-off



Processing time excludes participant decision time; approval percentage uses 18 executable cases.

Figure 6.4. Illustrative simulated processing-time and approval trade-off

Reference / predicted	Low	Medium	High	Total
Low	9	1	0	10
Medium	1	8	1	10
High	0	1	9	10
Total	10	10	10	30

Table 6.4. Illustrative simulated risk-classification confusion matrix

The illustrative comparison suggests the intended trade-off. Relative to a fixed approval gate, AutoOps AI improves unsafe-action prevention by 25.0 percentage points and verified remediation by 11.1 points while reducing approval requests from 18 to nine. Its median system-processing time is 1.4 seconds higher because it performs factor scoring, policy checks and verification. The confusion matrix shows that errors occur around the medium-risk boundary; critically, no simulated high-risk case is classified as low risk. These patterns would support the design rationale, but they cannot support the research conclusion until reproduced by the real prototype. Actual reporting must include raw n/N values, configuration versions, failed examples, reviewer agreement and confidence intervals where meaningful. Usability must remain unreported until genuine participants complete the approved procedure.

6.5 Chapter Summary

This chapter established the testing and evaluation process for AutoOps AI and demonstrated how its results should be analysed. The worked simulation illustrates reporting for retrieval, groundedness, risk classification, approval routing, unsafe-action prevention, verified remediation, recovery and efficiency. In that illustration, adaptive control improves safety and reduces approval burden at a small processing-time cost, while medium-risk classification remains the main calibration weakness. These values are not empirical findings. The final Chapter 6 must replace every simulated value and figure with outputs from the controlled experiment before Chapter 7 draws objective-by-objective conclusions.

Chapter – 07 CONCLUSIONS AND RECOMMENDATIONS

This research addressed the question of how a context-aware multi-agent IT-support system can combine RAG-grounded troubleshooting with risk-adaptive, human-gated and verifiable remediation. AutoOps AI was designed as a closed-loop framework rather than a general chatbot. A user reports a problem in natural language; specialised agents clarify the issue, retrieve approved organisational knowledge and form a structured diagnosis and action proposal. A separate deterministic safety layer then evaluates impact, diagnostic confidence, evidence quality, reversibility and the scope of affected resources. The resulting low, medium or high risk class informs whether the proposal may become an automatic candidate, requires user or IT-expert approval, or must be blocked and escalated.

The most important design decision is the separation of reasoning from authority. The language model can assist with understanding, retrieval and proposal generation, but it cannot grant itself execution permission. Actions must match an allow-listed catalogue entry and a policy decision. Any approval is

limited to the exact action, parameters, target, actor and expiry time. Preconditions examine the current state and permissions before execution. Postconditions test whether the expected technical and user-visible outcome was achieved. A command that runs without error is therefore not automatically treated as a resolved incident. If verification fails, the system attempts a safe rollback when available or escalates the case with its complete trace.

The controlled evaluation defined in Chapter 6 is intended to determine whether this design improves unsafe-action prevention, verified remediation and approval efficiency compared with simpler approaches. Special attention is required at the medium-risk boundary because the appropriate authority can change with user role, device ownership, reversibility and organisational policy. The final interpretation must therefore use actual executions, independent expert labels and retained failure traces. Until those data are collected, the thesis can conclude that the framework is technically feasible at prototype level, but it cannot claim a measured level of superiority.

7.1 Accomplishment of the Research Objectives

The objectives were examined through triangulation of literature findings, design artefacts, implementation evidence, deterministic tests, execution traces and the controlled comparative evaluation design. Table 7.1 distinguishes work already demonstrated by the thesis artefact from conclusions that still depend on the final experiment. This distinction avoids overstating the study while showing how each objective is addressed.

Objective	Evidence of accomplishment	Final status
O1: Identify limitations and risks	Chapter 1 defined delays and inconsistency in conventional support and the additional risks of hallucinated, unauthorised, irreversible or weakly evidenced agent actions.	Achieved through problem analysis and requirements identification.
O2: Analyse relevant research	Chapter 2 compared AIOps, multi-agent LLMs, RAG evaluation, runtime controls, human oversight and verifiable tool-use approaches, then established the combined research gap.	Achieved through the 2019-2026 literature review.
O3: Design and develop AutoOps AI	Chapters 4 and 5 specified and implemented specialised agents, evidence retrieval, five-factor risk scoring, approval routing, scoped tokens, pre/postconditions, audit events, rollback and escalation.	Achieved at controlled-prototype level; production deployment is outside scope.
O4: Evaluate safety and effectiveness	Chapter 6 defines ten principal cases, comparison conditions, metrics and trace requirements. Its current numerical dataset is illustrative.	Evaluation design completed; final empirical conclusion requires observed results and genuine usability data.

Table 7.1. Accomplishment of the research objectives

The first three objectives can be supported directly by the documented analysis, system specification and prototype evidence. The fourth objective is only fully accomplished when the experiment described in Chapter 6 is executed with a frozen configuration, expert reference labels and retained raw traces. The final objective by objective statement should then quote observed n/N values, report classification errors and discuss failures instead of reporting only percentages. Usability and trust conclusions must be based on genuine participants and the approved ethical process.

7.2 Problems Encountered

The first major challenge was converting a broad idea of intelligent IT support into a research contribution that could be implemented and measured. Conversational support, ticket management, RAG, multiple agents and automated actions could easily become a collection of loosely connected features. The project therefore centred its novelty on risk-adaptive and verifiable remediation while retaining the other components as supporting architecture. This focus made it possible to define measurable safety decisions, execution states and evaluation criteria.

A second challenge was controlling the uncertainty of LLM output. The same issue may be described in different language, and generated diagnoses can vary across runs. Directly connecting this output to PowerShell would make repeatability and safety difficult to defend. The solution was to require a typed ActionProposal, restrict execution to registered adapters and keep risk scoring and policy overrides deterministic. Nevertheless, diagnostic confidence and evidence quality still require careful operational definitions and calibration against expert judgement.

A third challenge concerned evidence quality. Semantic similarity does not guarantee that a retrieved instruction is current, approved or suitable for the user's environment. The RAG pipeline therefore stores metadata such as source, version, environment and approval state, while the controller treats missing, conflicting or outdated evidence as a reason to raise risk or stop execution. Creating a sufficiently varied but safe organisational knowledge corpus remains a limitation of the controlled prototype.

The fourth challenge was defining success. An execution API may return a successful status even though the original problem remains. AutoOps AI therefore separates EXECUTED from VERIFIED and requires postconditions. Some end-user problems have simple machine-checkable outcomes, whereas others require user confirmation or extended observation. Rollback is also action-specific and cannot always restore every external effect. These cases must be escalated rather than forced into a false success state.

7.3 Self-Reflection

7.3.1 Ideology of the Research

The research began from the practical expectation that AI could reduce repetitive support work. The investigation changed that assumption into a more cautious principle: useful autonomy is not the same as maximum autonomy. A safer system gives the AI only the authority justified by the current evidence, context and potential impact. Human involvement is not treated as a failure of automation; it is a control that should be applied selectively where uncertainty or consequence is higher.

7.3.2 Benefits Gained

The project strengthened skills in requirements analysis, multi-agent service design, RAG pipelines, API development, data modelling, rule based risk assessment, secure tool integration and interface communication. It also developed a clearer understanding of research traceability. A strong prototype is not demonstrated only by a working screen: claims must be connected to requirements, test cases, raw

outputs, metrics and limitations. The work further showed the importance of displaying evidence, risk explanations and action state without exposing private internal model reasoning.

7.3.3 Learning Curve

The main learning curve involved moving from prompt centred development to contract centred engineering. Early approaches could ask an LLM to decide what was safe, but that would leave authority inside an uncertain component. The final design uses schemas, allow lists, policy rules, scoped approvals and verification contracts around the model. Another important lesson was that safety evaluation must retain negative cases. Failed preconditions, denied approvals, classification errors, unsuccessful postconditions and rollbacks provide more useful evidence than a demonstration containing only successful scenarios.

7.4 Business Insight of AutoOps AI

AutoOps AI could create organisational value by reducing the time spent on repetitive, well understood support incidents while preserving expert attention for ambiguous or high impact cases. RAG can help staff and users apply approved internal guidance consistently. Risk adaptive routing can avoid sending every minor action through the same approval queue, while still protecting shared resources and privileged settings. Verification and structured audit records can improve accountability by showing which evidence, policy, approval and system state supported each decision.

The framework should be introduced as decision support and controlled remediation rather than as a replacement for an IT department. A practical adoption path would start with read only diagnosis and a small catalogue of reversible, user-local actions. Organisations could then expand automation only after observing classification quality, failure modes, rollback reliability and staff acceptance. Business measures may include verified first-contact resolution, median handling time, escalation appropriateness, avoided unsafe changes and expert time saved. Cost reduction should never be interpreted independently from safety and service quality.

7.5 Real-World Application Possibilities

The concept is applicable to university help desks, internal enterprise support, managed service providers and other environments with repeated device and account incidents. Suitable initial cases include clearing a user local cache, restarting an approved local service, repairing a known network configuration, collecting diagnostic information and applying a reversible application setting. The same control pattern may later support cloud operations or DevOps workflows, but the action catalogue, policy, identity model, verification contracts and escalation paths must be redesigned for those environments.

A production deployment would require integration with identity and access management, endpoint-management tools, configuration databases and established ticketing platforms. Organisational runbooks would need owners, review dates and version control. Security teams would define prohibited operations and critical overrides. IT experts would review borderline cases and failed postconditions. Monitoring would be necessary to detect policy drift, changes in retrieval quality and repeated misclassification. These controls are especially important because a laboratory prototype cannot represent every device state, dependency or organisational consequence.

7.6 Future Recommendations

Future work should extend AutoOps AI carefully without weakening the separation between AI reasoning and execution authority. The following recommendations are ordered from immediate research needs to longer-term extensions.

7.6.1 Complete the Empirical Evaluation and Calibrate Risk

The first priority is to execute the controlled experiment and replace every illustrative value in Chapter 6. A larger expert-labelled scenario set should be used to calibrate the risk thresholds. Particular attention should be given to medium risk cases and false safe errors because an unsafe low classification is more serious than an unnecessary escalation. Multiple reviewers should independently label a representative subset, with inter-rater agreement, raw n/N values, configuration versions and failure traces reported alongside percentages. An approved usability study should separately examine whether users and IT experts understand the evidence, risk explanation and approval request.

7.6.2 Expand the Action Catalogue Through Staged Assurance

Additional remediation actions should be introduced in stages. Each adapter should define permitted parameters, minimum privileges, preconditions, postconditions, rollback behaviour and escalation criteria before it becomes available. New actions should begin in recommendation only or dry run mode and progress to limited execution only after passing unit, integration, security and failure-injection tests. Actions affecting shared infrastructure, identity, security controls or irreversible data should remain expert-controlled or prohibited unless a strong organisational case and independent assurance exist.

7.6.3 Strengthen Knowledge and Agent Security

Future versions should add stronger protection against malicious or outdated knowledge. Recommended controls include signed and versioned runbooks, source ownership, review expiry rules, retrieval filters, conflict detection and independent validation of instructions before they influence an action. Adversarial testing should cover prompt injection, manipulated documents, tool-output injection, approval replay and attempts to change parameters after approval. The deterministic controller and executor must continue to treat retrieved text as evidence rather than executable authority.

7.6.4 Improve Explainability and Human Oversight

The approval interface should be evaluated and refined so that a decision-maker can quickly see the diagnosed problem, supporting sources, affected resources, expected benefit, possible harm, reversibility and verification plan. Explanations should remain concise and role-appropriate. The system should allow a user or expert to reject, modify or escalate a proposal without pressure to accept the AI recommendation. Future research can compare different explanation formats and measure decision accuracy, response time, perceived control and appropriate reliance.

7.6.5 Add Enterprise Governance and Continuous Monitoring

A production-oriented version should integrate organisational identity, role-based access, endpoint management, configuration records and ticketing platforms. Policies, weights, thresholds, action contracts and model configurations should be versioned so that past decisions can be replayed. Operational monitoring should detect changes in retrieval quality, repeated misclassification, abnormal approval

patterns, rollback failures and policy drift. Periodic review by IT operations and security staff should determine whether an action remains suitable for automation.

7.6.6 Extend Platforms and Conduct Longitudinal Evaluation

The prototype can later be extended beyond controlled Windows incidents to Linux, macOS, cloud and DevOps environments. Each environment requires its own action catalogue, identity model, preconditions, postconditions and recovery procedures; policies should not be copied without validation. Longer real world studies should measure verified first-contact resolution, handling time, unsafe action avoidance, escalation quality, expert workload, user trust and recurring failure patterns. Privacy preserving audit analysis and retention limits should be defined before operational logs are used for research.

7.6.7 Investigate Predictive Issue Detection as a Separate Extension

Predictive issue detection may be studied in future as a separate extension that identifies warning patterns before a user reports a problem. It would require suitable historical data, privacy and fairness assessment, drift monitoring and a new evaluation design. A prediction must never directly authorise remediation. It should create an observation or investigation request that passes through the same evidence, risk, approval and verification controls used for reported incidents. This separation preserves the present thesis focus on safe remediation after an issue is reported.

Priority	Recommended extension	Required safeguard or evidence
Near term	Complete real evaluation, calibrate risk thresholds, expand verified Windows adapters and conduct approved usability testing.	Frozen configuration, independent labels, raw traces, participant consent and transparent reporting.
Medium term	Strengthen knowledge security, approval explanations, enterprise identity, endpoint management and policy versioning.	Least privilege, signed sources, separation of duties, adversarial testing and continuous monitoring.
Long term	Evaluate cross platform and cloud remediation, longitudinal use and predictive issue detection as a separate extension.	Environment specific contracts, drift controls, new ethics review and independent evaluation.

Table 7.2. Recommended development and research roadmap

Additional research may examine uncertainty calibration, policy simulation before deployment, formal verification of selected high value adapters and privacy-preserving analysis of audit records. These extensions should retain the project's central safety boundary: neither an LLM response nor an automated signal should be sufficient to grant execution permission.

7.7 Final Summary

AutoOps AI demonstrates a structured approach to granting limited autonomy in IT support. Its contribution is the integration of context aware multi agent analysis and RAG-grounded troubleshooting with an independent five-factor risk assessment, graded human approval, pre-action validation, restricted execution, post-action verification, rollback, escalation and auditability. This architecture addresses the central weakness of blindly executing an AI generated fix: a plausible recommendation is not sufficient evidence of permission, safety or success.

The prototype and methodology support the feasibility of the framework. The controlled evaluation must now provide the empirical evidence needed to determine the extent to which AutoOps AI improves safe resolution, traceability and efficiency. The lasting principle of the research is that AI autonomy in IT operations should be conditional, explainable, reversible where possible and continuously verified.

REFERENCES

- [1] M. Shetty, C. Bansal, S. P. Upadhyayula, A. Radhakrishna, and A. Gupta, “AutoTSG: Learning and Synthesis for Incident Troubleshooting,” May 2022, [Online]. Available: <http://arxiv.org/abs/2205.13457>
- [2] S. Yao *et al.*, “REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS.” [Online]. Available: <https://react-lm.github.io/>.
- [3] Q. Wu *et al.*, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations.” [Online]. Available: <https://github.com/microsoft/autogen>
- [4] T. Guo *et al.*, “Large Language Model based Multi-Agents: A Survey of Progress and Challenges,” 2024.
- [5] Z. Wang *et al.*, “RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models,” in *International Conference on Information and Knowledge Management, Proceedings*, Association for Computing Machinery, Oct. 2024, pp. 4966–4974. doi: 10.1145/3627673.3680016.
- [6] S. Es, J. James, L. Espinosa-Anke, S. Schockaert, and E. Gradients, “RAGAS: Automated Evaluation of Retrieval Augmented Generation.” [Online]. Available: <https://platform.openai.com>
- [7] Y. Ruan *et al.*, “IDENTIFYING THE RISKS OF LM AGENTS WITH AN LM-EMULATED SANDBOX.”
- [8] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents.” [Online]. Available: <https://agentdojo.spylab.ai>
- [9] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents,” *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil*, vol. 1, 2026, doi: 10.1145/3744916.3764546.
- [10] E. Tabassi, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” Jan. 2023. doi: 10.6028/NIST.AI.100-1.
- [11] “Artificial intelligence risk management framework :,” Jul. 2024. doi: 10.6028/NIST.AI.600-1.
- [12] A. Doshi, Y. Hong, C. Xu, E. Kang, A. Kapravelos, and C. Kästner, “Towards Verifiably Safe Tool Use for LLM Agents,” in *Proceedings of the IEEE/ACM 48th International Conference on*

Software Engineering, New York, NY, USA: ACM, Apr. 2026, pp. 201–205. doi: 10.1145/3786582.3786839.

- [13] S. Amershi *et al.*, “Guidelines for human-AI interaction,” in *Conference on Human Factors in Computing Systems - Proceedings*, Association for Computing Machinery, May 2019. doi: 10.1145/3290605.3300233.
- [14] Y. Chen *et al.*, “AIOPSLAB: A HOLISTIC FRAMEWORK TO EVALUATE AI AGENTS FOR ENABLING AUTONOMOUS CLOUDS,” 2025. [Online]. Available: <https://aka.ms/aiopslab-repo>.
- [15] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” [Online]. Available: <https://github.com/huggingface/transformers/blob/master/>
- [16] Z. Xiang *et al.*, “GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning,” 2025. [Online]. Available: <https://guardagent.github.io/>
- [17] Z. Chen, M. Kang, and B. Li, “SHIELDAGENT: Shielding Agents via Verifiable Safety Policy Reasoning,” 2025.
- [18] Y. Liu *et al.*, “ToolGate: Contract-Grounded and Verified Tool Execution for LLMs.” [Online]. Available: <https://github.com/OceannTwT/>
- [19] Y. Mou *et al.*, “ToolSafe: Enhancing Tool Invocation Safety of LLM-based Agents via Proactive Step-level Guardrail and Feedback.”
- [20] E. Engström, M. A. Storey, P. Runeson, M. Höst, and M. T. Baldassarre, “How software engineering research aligns with design science: a review,” *Empir. Softw. Eng.*, vol. 25, no. 4, pp. 2630–2660, Jul. 2020, doi: 10.1007/s10664-020-09818-7.
- [21] M.-A. Storey, R. Hoda, A. M. P. Milani, and M. T. Baldassarre, “Guiding Principles for Using Mixed Methods Research in Software Engineering,” Mar. 2025, [Online]. Available: <http://arxiv.org/abs/2404.06011>
- [22] K. Schwaber and J. Sutherland, “The Scrum Guide The Definitive Guide to Scrum: The Rules of the Game,” 2020.